

OTH Regensburg

Audio-Based Bearing Fault Detection on Resource-Constrained Microcontrollers: An Ablation-Driven Study

Master's Thesis

Program: M.Sc. Electrical and Microsystems Engineering (MEM)

Author: Ramin Nassiri (**Matriculation No.:** 3313776)

Supervisor: Prof. Dr.-Ing. Armin Sehr

Second Examiner: Prof. Dr.-Ing. Johannes Reschke

Place/Date: Regensburg, 14 Aug 2025

Declaration of Authorship

1. I am aware that this copy of the thesis will transfer into the ownership of OTH Regensburg as piece of graded work.
2. I hereby declare that I wrote this thesis myself and did not use it elsewhere as graded work. I have used none, but the specified sources and tools and denoted all quotations as such.

Place, Date: Regensburg, 14 Aug 2025



Ramin Nassiri (Signature)

Erratum — Correction to FFT statements

Thesis: "Audio-Based Bearing Fault Detection on Resource-Constrained Microcontrollers: An Ablation-Driven Study" — submitted 14 Aug 2025
Author: Ramin Nassiri (Matric. 3313776) — Program: [MEM]

Scope of correction
The firmware **does** expose FFT via ulab.numpy.fft. In my on-device experiments I **did not use** it; I used a Python DFT fallback. Methods, data, results, and conclusions are **unchanged**. Using FFT would only **reduce runtime** of spectral feature extraction at the last test run.

Corrections (Old → New)

- **Abstract** (p. 2)
  Old: "…spectral feature extraction dominated latency **without a native FFT/IIR path**."
  New: "…spectral feature extraction dominated latency **because FFT was not used in these runs** (the firmware exposes FFT as ulab.numpy.fft)."
- **Chapter 4.1 — Device/firmware paragraph** (p. 33)
  Old: "The build included ulab **but did not provide ulab.fft; therefore,** spectral features used a Python DFT fallback on-device."
  New: "The build includes ulab and **exposes FFT via ulab.numpy.fft; however, it was not used** in my runs, so a Python DFT fallback was used on-device."
- **Chapter 4.3 — Feature Stack Implemented On-Device (A8)** (p. 34)
  Old: "**Because ulab.fft was unavailable in this firmware**, a Python DFT was used …"
  New: "**Although FFT is available via ulab.numpy.fft, it was not used**; therefore a Python DFT was used …"
- **Listing 4.1 note** (p. 36)
  Old: "A native FFT (**ulab.fft** or C) would reduce latency substantially."
  New: "A native FFT (**ulab.numpy.fft** or C) would reduce latency substantially."
- **Chapter 4.7 — Comparison to Vibration Pipeline** (p. 36)
  Old: "…**without a native FFT**, the Python DFT dominates runtime."
  New: "…**since FFT wasn't used**, the Python DFT dominated runtime."
- **Chapter 5 — Summary of Findings** (p. 38)
  Old: "…feature extraction dominated latency **due to the absence of a native FFT**."
  New: "…feature extraction dominated latency **because FFT wasn't used in our runs**."
- **Chapter 5 — Practical Guidance** (p. 39)
  Old: "On-device pipeline: enable a real FFT (via **ulab.fft** or a small C FFT) …"
  New: "On-device pipeline: enable a real FFT (via **ulab.numpy.fft** or a small C FFT) …"
- **Chapter 5 — Future Work #1** (p. 40)
  Old: "**Upgrade to a MicroPython build with ulab.fft** …"
  New: "**Use the available ulab.numpy.fft (or ensure it's enabled)** or integrate C-level FFT/IIR …"

Note: This erratum corrects **wording about firmware capability only**. No methodological changes; all figures, tables, and conclusions remain valid.

Abstract

This thesis investigates whether *acoustic* signals can support reliable bearing fault detection under the constraints of resource-limited microcontrollers. A systematic ablation study was conducted over preprocessing (sampling rate, narrow band-pass filtering, Hilbert envelope and logarithmic compression), scaling, dimensionality reduction (PCA), lightweight classifiers (Gaussian Mixture Models and small Multi-Layer Perceptrons), and segment- vs. file-level aggregation (majority, median log-likelihood, mean probability). Experiments used a publicly available bearing-sound dataset recorded with a laboratory microphone under controlled operating conditions.

The ablation identifies compact pipelines that balance accuracy and deployability. Narrowband features ($\approx$ 1–5 kHz) with envelope+log compression and PCA ($k \leq 8$) consistently outperform wider or uncompressed alternatives. The best MLP pipeline (A16: 8 kHz, 1–3.5 kHz band-pass, envelope+log, PCA(8), MinMax) reached **F1** = 0.923 with **Acc.** = 0.889, while the strongest GMM configuration (A8 + per-file aggregation) achieved **F1** = 0.909 and **Acc.** = 0.889. File-level aggregation reliably improved stability over segment-level predictions.

Embedded feasibility was examined on an ESP32-based M5Stack Core2 using MicroPython. PCA+GMM inference executed in tens of milliseconds per call, whereas spectral feature extraction dominated latency without a native FFT/IIR path. Consequently, the bottleneck lies in the front end rather than the classifier. Practical microphone considerations are discussed; given the gap between laboratory microphones and typical MEMS devices, file-based evaluation was used to avoid sensor-limited conclusions.

Overall, it is shown that audio-based bearing diagnostics are compatible with TinyML constraints when compact features and lightweight models are used. Real-time deployment becomes practical with accelerated DSP kernels and an appropriate microphone chain, extending the lab's vibration-centric PDMS toward a complementary acoustic modality.

Table of Contents

Abstract **2**

List of Figures **6**

List of Tables **8**

List of Listings **9**

1 Introduction **10**
1.1 Context: Edge AI under Resource Constraints 10
1.2 Bearings and Fault Diagnostics (Technical Background) 10
1.3 Sensing Modalities: Vibration vs. Audio 11
1.4 Bearings, Faults, and Condition Monitoring 11
1.5 Why Audio on Tiny Devices 12
1.6 Background: SES–PDMS on ESP32-Class Hardware 12
1.7 Problem Statement and Research Questions 12
1.8 Approach and Dataset 13
1.9 Contributions 13
1.10 Scope and Limitations 14
1.11 Thesis Organization 14

2 Audio Analysis **15**
2.1 Dataset Selection 15
2.2 Data Preparation and Conversion 15
2.2.1 Representative Conversion and Segmentation Script 16
2.2.2 Model Selection: Gaussian Mixture Model (GMM) and Multi-Layer Perceptron (MLP) 17
2.2.3 Feature Extraction Strategy 18

3 Ablation Study and Experimental Evaluation **20**
3.1 Overview and Rationale 20

3.2 Experimental Setup 20
3.2.1 Data and Base Pipeline 20
3.2.2 Models and Metrics 20
3.3 Parameters Investigated 21
3.3.1 Sampling Rate 21
3.3.2 Filtering and Spectral Shaping 21
3.3.3 Feature Representations 22
3.3.4 Normalization and Dimensionality Reduction 22
3.3.5 Temporal Aggregation (Per-File Decisions) 22
3.4 Methodology 23
3.4.1 Confusion Matrix as an Evaluation Tool 23
3.5 Results 23
3.5.1 Global Trends and Summary Figures 23
3.5.2 Parameter Effects and Trends 24
3.5.3 Top and Bottom Performers 28
3.5.4 Representative Confusion Matrices (Grouped) 30
3.6 Discussion 31
3.7 Reproducibility and Availability 32

4 On-Device Inference on ESP32 (M5Stack Core2) 33
4.1 Why A8→GMM for Embedded 33
4.2 I/O Mode: WAV From Flash (Not Live Mic) 33
4.3 Feature Stack Implemented On-Device (A8) 34
4.4 Preprocess + Classifier Runtime 34
4.5 What Worked and What Did Not 34
4.6 How It Was Run (Commands and Code) 34
4.7 Comparison to Vibration Pipeline 36
4.8 Custom MicroPython Firmware: Rebuild, Flashing, and Verification (M5Stack Core2) 36
4.8.1 Target, Versions, and Goal 36
4.8.2 Key Pitfall and Resolution (ESP-IDF Mismatch) 37
4.8.3 Builtin Module Registration 37
4.8.4 Build, Merge, and Flash 37
4.8.5 Implication for Acceleration with `emlearn` 37

5 Conclusion 38

A Confusion Matrices of Ablation Experiments 41

B Microphone Modalities and Capture Choice **48**
B.1 PDM vs. PCM Microphones (Brief) 48
B.2 M5Stack Core2 Internal Mic (PDM) and MicroPython 48
B.3 Reference Dataset Microphone (uOttawa) vs. M5Stack Options 49

C Ablation Runs (Key Parameters) **50**

Bibliography **52**

List of Figures

3.1 Generic 2×2 confusion matrix layout. 23
3.2 File-level F1 across ablation runs (A0–A41). 24
3.3 Top three configurations by file-level F1. 24
3.4 Aggregation comparison (mean vs. median vs. majority) for representative best models. 24
3.5 Impact of scaling/PCA families (B0–B2) on file-level F1. 25
3.6 Consolidated best-of summary (reference image). 25
3.7 Top 15 runs by file-level F1. 26
3.8 Mean F1 by aggregation method over file-level runs. 26
3.9 F1 distribution by sampling rate (all models). 26
3.10 F1 distribution by sampling rate (MLP only). 27
3.11 F1 distribution by sampling rate (GMM only). 27
3.12 Band-pass effect on mean F1 (MLP). 27
3.13 Band-pass effect on mean F1 (GMM). 28
3.14 Mean F1 vs PCA components (MLP). 28
3.15 Mean F1 vs PCA components (GMM). 28
3.16 Largest improvements from file-level aggregation (ΔF1). 29
3.17 F1 distributions at segment vs file levels (where both available). . . . 29
3.18 Baselines: comparison of class-wise performance (A0). 30
3.19 Representative improvements with BP+Hilbert(+log) pipelines. 30
3.20 Per-file aggregation reduces frame-level volatility and improves stability. 31
3.21 Parameter changes that increased FP (GMM) and FN (MLP). 31

A.1 A0 confusion matrices: GMM (left) and MLP (right). 41
A.2 A1 confusion matrices: GMM (left) and MLP (right). 42
A.3 A2 confusion matrices: GMM (left) and MLP (right). 42
A.4 A3 confusion matrices: GMM (left) and MLP (right). 42
A.5 A4 confusion matrices: GMM (left) and MLP (right). 42
A.6 A5 confusion matrices: GMM (left) and MLP (right). 43
A.7 A6 confusion matrices: GMM (left) and MLP (right). 43

A.8 A7 confusion matrices: GMM (left) and MLP (right). 43
A.9 A8 confusion matrices: GMM (left) and MLP (right). 43
A.10 A14 confusion matrices: GMM (left) and MLP (right). 44
A.11 A16 confusion matrices: GMM (left) and MLP (right). 44
A.12 A17 confusion matrices: GMM (left) and MLP (right). 44
A.13 A18 confusion matrices: GMM (left) and MLP (right). 44
A.14 A19 confusion matrices: GMM (left) and MLP (right). 45
A.15 A20 confusion matrices: GMM (left) and MLP (right). 45
A.16 A21 confusion matrices: GMM (left) and MLP (right). 45
A.17 A22 confusion matrices: GMM (left) and MLP (right). 45
A.18 A23 confusion matrices: GMM (left) and MLP (right). 46
A.19 A24 confusion matrices: GMM (left) and MLP (right). 46
A.20 A25 confusion matrices: GMM (left) and MLP (right). 46

List of Tables

2.1 SES Laboratory 25-feature set for PDMS (adapted for audio signals) . 19

3.1 Top configurations by file-level F1 (one best run per ID). 29
3.2 Lowest file-level F1 configurations (from `ablation_clean.csv`). . . 30

A.1 Band-pass effect on mean F1 (MLP). 47
A.2 Band-pass effect on mean F1 (GMM). 47
A.3 Mean F1 by aggregation method (file-level runs). 47

B.1 Capture options referenced in this thesis (indicative specs). 49

C.1 Ablation variants: run name, model, and headline metrics. 51

List of Listings

2.1 Representative data conversion, segmentation, and splitting 16
3.1 Band-pass + Hilbert envelope with optional log compression 21
3.2 Normalization and PCA reduction 22
3.3 Aggregation from frame to file level 22
4.1 Device inference: load WAV from flash, extract A8, preprocess, run GMM. 35

Chapter 1

Introduction

1.1 Context: Edge AI under Resource Constraints

Edge computing moves data processing from cloud servers to the devices that sense the world. When inference executes next to the sensor—often termed *Edge AI*—latency becomes predictable, bandwidth needs shrink, and sensitive data can remain on-site. Microcontroller-based systems (MCUs) make this practical: a few hundred kilobytes to a few megabytes of RAM and flash, low-power operation, and robust I/O allow continuous monitoring in industrial settings. The constraint is tight resources, which pushes designs toward efficient feature front-ends and compact models [1, 2]. Toolchains such as TensorFlow Lite Micro and public benchmarks like MLPerf Tiny demonstrate that useful accuracy and latency can be achieved within MCU budgets for tasks such as keyword spotting, anomaly detection, and tiny image classification [3, 4]. These patterns motivate similar approaches for industrial condition monitoring.

1.2 Bearings and Fault Diagnostics (Technical Background)

Rolling-element bearings consist of inner and outer races, rolling elements (balls or rollers), and a cage. Faults typically initiate at stress concentrators on the inner/outer raceways or rolling elements and evolve from microscopic cracks to macroscopic spalls. Their dynamics generate repetitive impact trains and amplitude/phase modulation in the measured signal, which appear as characteristic bands and sidebands in the spectrum. Classical vibration-based diagnostics therefore rely on demodulation and band-selection methods such as high-frequency envelope analysis and spectral kurtosis, often combined with order tracking under speed variation [5–7]. Reviews emphasize that detection quality depends not only on the algorithm, but also on sensor placement,

mounting, bandwidth, and the operating point of the machine [8, 9].

1.3 Sensing Modalities: Vibration vs. Audio

Two non-exclusive sensing routes exist for bearing condition monitoring:

Vibration (contact). An accelerometer mounted near the bearing provides a mechanically coupled, band-limited signal with high repeatability. Advantages include strong signal-to-noise ratio (SNR) for impulsive faults, mature processing workflows (envelope analysis, spectral kurtosis), and relative robustness to ambient acoustics. Limitations include installation effort (mounting, cabling), possible sensitivity to mounting quality, and the need for contact access to the asset [5, 7].

Audio (non-contact). A microphone captures airborne sound from the bearing and surrounding structure. Advantages include non-contact retrofitting, safety (no moving-part contact), and the ability to monitor enclosed or hot components. Limitations include dependence on microphone quality and positioning, reflections/reverberation, and contamination by ambient noise; as a result, careful band selection and envelope/log compression become important [8, 9]. In embedded settings, audio front-ends can be compute-heavy when implemented without a native FFT, so narrow band-pass filtering, envelope extraction, and low-dimensional features are preferred under TinyML constraints [2, 10].

Implications for TinyML on MCUs. Vibration streams are typically lower-bandwidth and easier to demodulate with inexpensive digital filters, which aligns well with microcontroller memory/latency limits. Audio offers simpler deployment but demands more from the spectral front-end to achieve comparable robustness. These trade-offs motivate the approach taken in this thesis: engineer compact audio features (e.g., narrow band-powers with envelope/log compression), reduce dimensionality (e.g., PCA), and pair with lightweight classifiers (GMM/MLP) that are feasible on ESP32-class hardware [3, 4, 11]. Practical microphone considerations and the rationale for file-based evaluation in this work are summarized in Appendix B.

1.4 Bearings, Faults, and Condition Monitoring

Rolling-element bearings are critical in rotating machinery, and their failures can lead to downtime, secondary damage, and safety risks. Faults often originate at the inner race, outer race, rolling elements, or cage, progressing from microscopic defects

to macroscopic damage. Classical vibration-based diagnostics exploit fault-related modulation and repetitive impact signatures; established techniques include envelope analysis, spectral kurtosis, and order tracking [5–7]. Surveys emphasize that effective detection hinges on the sensing modality, sensor quality and placement, and robust signal processing that tolerates noise, load variation, and speed fluctuations [8, 9].

1.5 Why Audio on Tiny Devices

Acoustic sensing is attractive because it does not require mechanical coupling and can be retrofitted easily. However, it is sensitive to microphone quality, distance, reflections, and environmental noise. From an embedded perspective, *front-end efficiency* is crucial: transforms and filters dominate runtime, hence designs with narrow band-pass filtering, envelope extraction, low-dimension features, and lightweight classifiers are preferred on MCUs [2, 10]. TinyML practice further suggests that carefully engineered features often outperform end-to-end deep models when memory and compute are limited [11].

1.6 Background: SES–PDMS on ESP32-Class Hardware

At OTH Regensburg, the Smart Embedded Systems (SES) Laboratory has developed a *Predictive Diagnostics and Maintenance System* (PDMS) targeting low-cost embedded platforms. Prior SES theses describe a modular stack on ESP32 hardware using MicroPython and `ulab`, reusable feature libraries, and GUI/data collection workflows across M5Stack devices [12]. Within this PDMS, compact feature vectors ($\sim$25 D) with *Gaussian Mixture Models* (GMM) and *Multi-Layer Perceptrons* (MLP) have repeatedly offered a strong balance of accuracy and deployability under MCU constraints; library work on 1D CNNs has also been explored for future acceleration [12]. To date, the SES pipeline has been vibration-centric; extending it to *audio* is novel within this context.

1.7 Problem Statement and Research Questions

Given the constraints of ESP32-class devices and the practical advantages of non-contact sensing, the central question is whether *acoustic* signals can support reliable bearing fault detection using MCU-feasible pipelines.

RQ1. Which preprocessing, features, and model choices (e.g., band-pass ranges, envelope/log compression, scaling, PCA, GMM vs. MLP) provide the best discriminability for healthy vs. faulty bearings from audio on this dataset?
RQ2. How do aggregation strategies (segment vs. per-file mean/median/-majority) affect stability and accuracy?
RQ3. Which pipelines remain practical for embedded inference on ESP32 (runtime, memory), and what are the bottlenecks?
RQ4. How do the benefits and limitations of audio compare to the lab's vibration-based PDMS baselines?

1.8 Approach and Dataset

A systematic ablation study was conducted. Each run modified exactly one parameter (sampling rate, filtering, envelope/log compression, scaling, PCA dimensionality, model type, or aggregation) while all others were held constant, and both GMM and MLP were evaluated. The experiments used the University of Ottawa bearing dataset's audio recordings (healthy vs. faulty) captured with a laboratory microphone and controlled operating conditions; the dataset provides high-quality acoustic evidence for benchmarking feature pipelines [13]. The ablation results identify a small set of consistently strong pipelines, which were then evaluated for on-device feasibility.

1.9 Contributions

The thesis offers three main contributions:

1. **Ablation-driven audio pipeline analysis.** A broad ablation across filters, features, scaling, PCA, models, and aggregation provides quantitative insight into which choices matter most for acoustic bearing fault detection under TinyML constraints.

2. **Compact, deployable pipelines.** Narrow-band, envelope/log features with PCA($k \leq 8$) and lightweight classifiers (GMM/MLP) are shown to achieve strong performance while remaining compatible with ESP32 memory and runtime budgets, in line with TinyML best practices [3, 4, 11].

3. **On-device validation.** A best-performing GMM pipeline was implemented on an M5Stack Core2 (ESP32) with MicroPython, exercising end-to-end inference from WAV files and documenting the practical bottlenecks (notably the spectral front-end), thereby informing future firmware and kernel choices.

1.10 Scope and Limitations

This study focuses on binary classification (healthy vs. faulty) under fixed operating conditions from the chosen dataset. Real-time live capture on the Core2's internal PDM microphone was out of scope due to firmware support constraints; high-quality WAV input was preferred to avoid sensor-quality bottlenecks during the ablation. Extension to multi-class fault types, variable speeds/loads, and live I^2S capture is left for future work. The design choices emphasize MCU feasibility over model capacity, consistent with TinyML constraints [2, 11].

1.11 Thesis Organization

Chapter 2 introduces the audio data preparation and feature front-end. Chapter 3 reports the ablation design and results, with representative confusion matrices and trend analyses. Chapter 4 details embedded implementation on ESP32, including timing, memory, and firmware considerations. The Appendix provides microphone modality background and rationale for file-based evaluation and includes the complete set of confusion matrices.

Chapter 2

Audio Analysis

2.1 Dataset Selection

To obtain viable recordings of both healthy and defective bearing acoustics, an open-access dataset from the University of Ottawa, referred to as UORED-VAFCLS / UODS-VAFDC, was employed. The dataset comprises acoustic and vibration signals recorded under controlled laboratory conditions with constant load and speed. Recordings cover 20 bearings across three health states (healthy, developing fault, faulty), producing 60 sessions at a consistent sampling rate of 42 kHz and 10 s duration per session. Acquisition utilized high-quality sensors (microphone, accelerometer, hall sensor, load cell), with raw data provided in CSV/XLSX/MAT formats and precomputed spectrograms (STFT with Hanning windows) supplied as images. This resource provides a robust basis for benchmarking acoustic diagnostics prior to adaptation to embedded microsystems such as ESP32 [13].

2.2 Data Preparation and Conversion

After download, the acoustic recordings were not available as `.wav` files and were therefore converted to 16-bit PCM WAV at 42 kHz to match the dataset specifications. The conversion yielded 20 healthy and 40 anomalous audio tracks, each 10 s in duration. Subsequent pre-processing followed standard audio ML practice: (i) format conversion, (ii) amplitude normalization and resampling (if required), (iii) segmentation into fixed-length windows, and (iv) division into training, validation, and test subsets to avoid leakage and to enable reliable evaluation.

2.2.1 Representative Conversion and Segmentation Script

Listing 2.1 illustrates a representative workflow used for conversion, segmentation, and dataset splitting. It demonstrates exporting 16-bit mono WAV at 42 kHz, peak normalization, fixed-window segmentation, and a 70/15/15 split.

```
import os
from pathlib import Path
import random
import shutil
from pydub import AudioSegment
import librosa
import soundfile as sf

# Directories
SRC = Path("raw_download")
WAV = Path("wav_converted")
SEG = Path("segments")
for d in (WAV, SEG):
    d.mkdir(parents=True, exist_ok=True)

# 1) Convert to 16-bit mono WAV @ 42 kHz
for f in SRC.glob("*.*"):
    audio = AudioSegment.from_file(f)
    audio = audio.set_frame_rate(42000).set_channels(1).set_sample_width
        (2)
    audio.export(WAV / (f.stem + ".wav"), format="wav")

# 2) Segmentation (e.g., 2-second windows) + peak normalization
SEG_DUR_S = 2.0
for f in WAV.glob("*.wav"):
    y, sr = librosa.load(f, sr=None, mono=True)
    peak = max(1e-9, abs(y).max())
    y = y / peak
    seg_len = int(SEG_DUR_S * sr)
    n_seg = len(y) // seg_len
    for i in range(n_seg):
        s = i * seg_len
        e = s + seg_len
        seg = y[s:e]
        out = SEG / f"{f.stem}_seg{i+1:04d}.wav"
        sf.write(out, seg, sr, subtype="PCM_16")
```

```
# 3) Train/Val/Test split (70/15/15)
all_wavs = list(SEG.glob("*.wav"))
random.shuffle(all_wavs)
n = len(all_wavs)
n_train = int(0.70 * n)
n_val = int(0.85 * n)

splits = {
    "train": all_wavs[:n_train],
    "val": all_wavs[n_train:n_val],
    "test": all_wavs[n_val:]
}
for split, files in splits.items():
    outdir = SEG / split
    outdir.mkdir(exist_ok=True)
    for f in files:
        shutil.move(str(f), outdir / f.name)
```

Listing 2.1: Representative data conversion, segmentation, and splitting

2.2.2 Model Selection: Gaussian Mixture Model (GMM) and Multi-Layer Perceptron (MLP)

In order to leverage established SES Laboratory experience and reduce risk in the initial phase, model development was focused on two proven approaches: the *Gaussian Mixture Model* (GMM) and the *Multi-Layer Perceptron* (MLP). Both models have repeatedly demonstrated strong performance for vibration-based bearing diagnostics on ESP32-class hardware in the PDMS line of work [12, 14, 15]. Transferring these baselines to the acoustic modality enables a controlled comparison while maintaining feasibility for on-device deployment.

Gaussian Mixture Model (GMM). A GMM models the data distribution as a mixture of Gaussian components whose parameters are typically estimated via the Expectation–Maximization (EM) algorithm. For anomaly detection, the likelihood under the fitted mixture serves as a decision statistic; segments with low likelihood are flagged as abnormal. In the SES PDMS context, GMMs have been employed effectively in unsupervised settings, reducing reliance on labeled data and supporting incremental adaptation on embedded targets [12, 14].

Multi-Layer Perceptron (MLP). An MLP is a supervised feedforward neural network comprising an input layer, one or more hidden layers with non-linear activations, and an output layer. With hand-crafted feature vectors (time/frequency/autocorrelation domains), MLPs have achieved high classification accuracy in prior SES studies while remaining computationally tractable on ESP32-based systems [12, 15]. In this work, analogous feature representations derived from audio are used to enable a fair modality comparison and to preserve deployability on resource-constrained hardware.

2.2.3 Feature Extraction Strategy

Previous work in the SES Laboratory on vibration-based predictive maintenance systems (PDMS) has established a standardized feature extraction pipeline comprising 25 statistical and spectral features. This feature set, derived from time-domain, frequency-domain, and autocorrelation analyses, has consistently delivered high classification performance for bearing fault detection on embedded platforms [12, 14, 15].

In this project, the same 25-feature methodology was applied to acoustic signals, with the objective of evaluating whether the proven vibration-based approach could be successfully adapted to sound-based diagnostics. By maintaining the feature definition while changing the sensing modality, it becomes possible to directly compare the performance of machine learning models across vibration and acoustic domains under otherwise similar processing and deployment conditions.

Table 2.1: SES Laboratory 25-feature set for PDMS (adapted for audio signals)

#	**Domain**	**Feature Name**
1	Time	Mean
2	Time	Standard deviation
3	Time	Variance
4	Time	Skewness
5	Time	Kurtosis
6	Time	Root mean square (RMS)
7	Time	Peak value
8	Time	Crest factor (Peak/RMS)
9	Time	Shape factor (RMS/Mean absolute value)
10	Time	Impulse factor (Peak/Mean absolute value)
11	Frequency	Spectral centroid
12	Frequency	Spectral spread
13	Frequency	Spectral skewness
14	Frequency	Spectral kurtosis
15	Frequency	Spectral flatness
16	Frequency	Spectral roll-off (e.g., 85%)
17	Frequency	Spectral entropy
18	Frequency	Dominant frequency
19	Frequency	Bandwidth
20	Frequency	Energy in specific band (e.g., 1–3 kHz)
21	Autocorrelation	Maximum autocorrelation (lag > 0)
22	Autocorrelation	Lag of maximum autocorrelation
23	Autocorrelation	Zero-crossing rate
24	Autocorrelation	Autocorrelation energy
25	Autocorrelation	Autocorrelation entropy

Chapter 3

Ablation Study and Experimental Evaluation

3.1 Overview and Rationale

The ablation study constitutes the central experimental pillar of this work. Its purpose is to isolate and quantify the effect of individual pipeline decisions on acoustic bearing classification. In each controlled run, exactly one parameter (e.g., sampling rate, filter, normalization, dimensionality reduction, or aggregation method) was changed while all other settings were held constant. Two classifiers were evaluated throughout: a *Gaussian Mixture Model* (GMM) and a *Multi-Layer Perceptron* (MLP), reflecting prior SES Lab practice in vibration-based PDMS research [12, 14, 15]. Acoustic data originate from the University of Ottawa bearing-sound dataset [13].

3.2 Experimental Setup

3.2.1 Data and Base Pipeline

Recordings were converted to 16-bit PCM WAV and segmented into fixed-duration frames. The SES Laboratory 25-feature pipeline (time/frequency/autocorrelation domains) was adapted to audio. Unless stated otherwise, features were normalized and optionally reduced with principal component analysis (PCA) before classification. Train/validation/test partitions were used for fitting, selection, and reporting.

3.2.2 Models and Metrics

The study employed: (i) an unsupervised GMM (diagonal covariance unless specified) and (ii) a compact MLP suitable for embedded deployment [12, 15]. Metrics include

Accuracy, Precision, Recall, and F1-score; F1 is emphasized due to potential class imbalance. Representative confusion matrices are shown to reveal class-wise behavior beyond aggregate scores.

3.3 Parameters Investigated

Parameters were derived from the ablation logs and are grouped below with references to standard DSP/ML literature.

3.3.1 Sampling Rate

Two rates were explored: 8 kHz and 16 kHz. Lower rates reduce computational load and can suppress high-frequency noise, whereas higher rates preserve more harmonics and transient detail at the cost of increased dimensionality [10]. Optimal choice is task- and sensor-dependent.

3.3.2 Filtering and Spectral Shaping

High-pass filtering (HPF) near 100 Hz mitigates DC drift and very low-frequency energy. **Notch filtering** at 50/60 Hz removes mains interference [10]. **Band-pass** windows (1–3.5 kHz, 1–5 kHz) target bands where bearing fault acoustics concentrate [8]. **Hilbert envelope** extraction (magnitude of the analytic signal) captures amplitude-modulated, impulsive signatures; **log compression** stabilizes dynamic range [16]. **Spectral whitening** flattens the spectrum to highlight narrow-band deviations [10].

Token code: Band-pass + Hilbert envelope (+ optional log)

```
import numpy as np
from scipy.signal import butter, filtfilt, hilbert

def bandpass(x, sr, lo=1000.0, hi=5000.0, order=4):
    b, a = butter(order, [lo/(sr/2), hi/(sr/2)], btype='band')
    return filtfilt(b, a, x)

def preprocess_env_log(x, sr, lo=1000, hi=5000, use_log=True, eps=1e-9)
    :
    x_bp = bandpass(x, sr, lo, hi)
    x_env = np.abs(hilbert(x_bp))
    return np.log(np.maximum(x_env, eps)) if use_log else x_env
```

Listing 3.1: Band-pass + Hilbert envelope with optional log compression

3.3.3 Feature Representations

SES 25 features: time/frequency/autocorrelation descriptors established in SES Lab vibration PDMS [14, 15]. **Log-Mel filterbanks** (20/40 bands) with summary statistics emphasize perceptually relevant bands [16]. **MFCCs** with or without deltas summarize the spectral envelope; widely applied to machinery acoustics [16]. **Log compression** reduces dynamic range and stabilizes variances.

3.3.4 Normalization and Dimensionality Reduction

MinMax and **z-score** scaling standardize ranges; **Robust** scaling mitigates outliers. Percentile-based **log normalization** (*norm90/95/99*) stabilizes heavy tails. **PCA(k)** reduces dimensionality while retaining variance and often improves GMM stability and generalization of compact MLPs [17].

Token code: Percentile normalization, z-score, PCA

```
import numpy as np
from sklearn.decomposition import PCA

def percentile_norm(x, p=95):
    s = np.percentile(np.abs(x), p)
    return x / max(s, 1e-9)

def zscore(x, axis=0, eps=1e-9):
    mu = np.mean(x, axis=axis, keepdims=True)
    sd = np.std(x, axis=axis, keepdims=True)
    return (x - mu) / np.maximum(sd, eps)

def pca_reduce(X, k=8):
    return PCA(n_components=k, svd_solver="auto").fit_transform(X)
```

Listing 3.2: Normalization and PCA reduction

3.3.5 Temporal Aggregation (Per-File Decisions)

Frame-level predictions were aggregated using **mean**, **median**, or **majority vote**. Mean aggregation averaged calibrated probabilities and typically yielded the most stable outcome; median is robust to outliers; majority emphasizes hard consensus.

Token code: Mean/median/majority aggregation

```
import numpy as np
from collections import Counter
```

```
def agg_mean(probs): # probs: (frames, classes)
    return np.argmax(probs.mean(axis=0))

def agg_median(probs):
    return np.argmax(np.median(probs, axis=0))

def agg_majority(preds): # preds: (frames,)
    return Counter(preds).most_common(1)[0][0]
```

Listing 3.3: Aggregation from frame to file level

3.4 Methodology

Each ablation modified exactly one parameter while holding all others fixed. For each configuration, both GMM and MLP were trained and evaluated. Performance was computed at the *segment level* and aggregated to the *file level*. Confusion matrices were produced for all runs to support class-wise interpretation.

3.4.1 Confusion Matrix as an Evaluation Tool

A confusion matrix tabulates true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), and provides richer insight than accuracy, especially with class imbalance [18].

		Predicted
Actual	Positive	Negative
Positive	TP	FN
Negative	FP	TN

Figure 3.1: Generic 2×2 confusion matrix layout.

3.5 Results

3.5.1 Global Trends and Summary Figures

Figure 3.2 shows file-level F1 across A0–A41; top-3 runs are highlighted in Figure 3.3. Aggregation comparison is shown in Figure 3.4; scaling/PCA family effects appear in Figure 3.5. A consolidated best-of image is included in Figure 3.6. The 15 strongest runs by F1 are depicted in Figure 3.7.

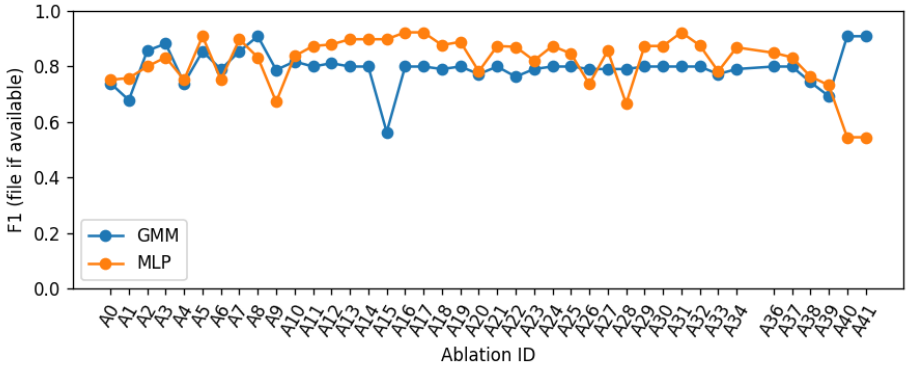


Figure 3.2: File-level F1 across ablation runs (A0–A41).

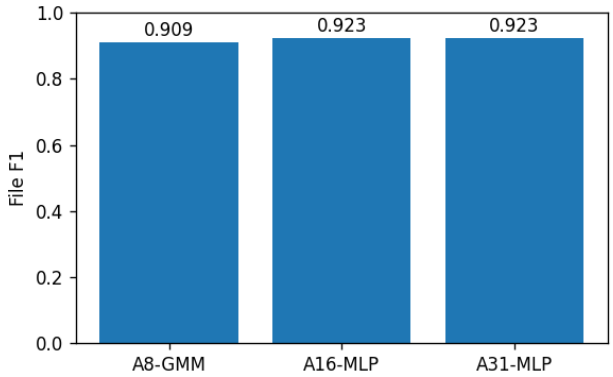


Figure 3.3: Top three configurations by file-level F1.

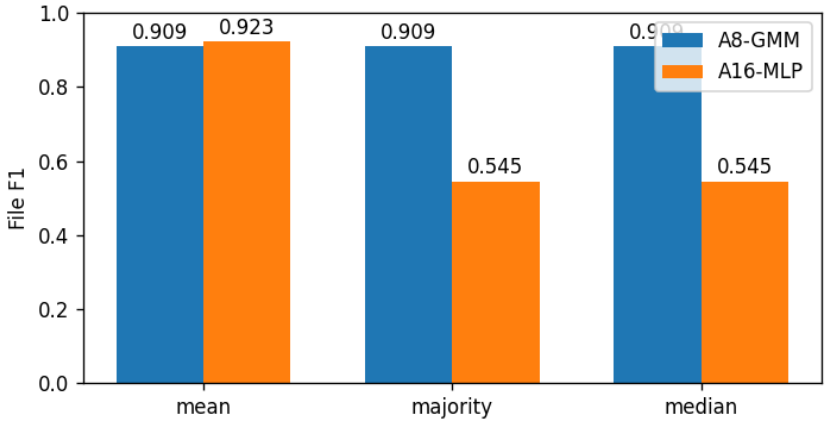


Figure 3.4: Aggregation comparison (mean vs. median vs. majority) for representative best models.

3.5.2 Parameter Effects and Trends

Aggregation (file-level). Across file-level runs, mean aggregation achieved a markedly higher average F1 (≈ 0.923) than median or majority (≈ 0.727), confirming the benefit of averaging calibrated probabilities (Figure 3.8).

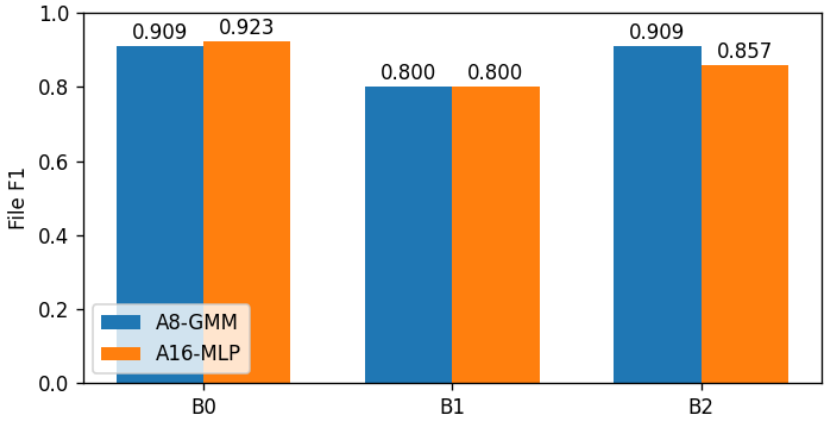


Figure 3.5: Impact of scaling/PCA families (B0–B2) on file-level F1.

label	id	file_F1	segment_F1	scaler	pca	notes
A16-MLP (env+log@8k)	A16-MLP-FILE	0.923076923				Per-file aggregation
A31-MLP (env+log@16k)	A31-MLP-FILE	0.923076923				Per-file aggregation
A8-GMM (BP1–5k@16k)	A8-GMM-FILE	0.909090909				Per-file aggregation
RF(50,depth≤8) on A16	A42-RF	0.7899159663865546				
ET(50,depth≤8) on A16	A43-ET	0.819672131147541				

Figure 3.6: Consolidated best-of summary (reference image).

Sampling rate. Overall, 8 kHz yielded higher mean F1 than 16 kHz across all models (0.833 vs 0.814). Split by model, MLP benefited from 8 kHz (0.900 vs 0.836), while GMM slightly favored 16 kHz (0.792 vs 0.767). See Figure 3.9–3.11.

Band-pass window. For MLP, the 1–3.5 kHz band produced the highest mean F1 ($\approx$ 0.900), followed by 1–5 kHz ($\approx$ 0.839), and no band-pass ($\approx$ 0.782), indicating that a narrower, fault-rich band improves separability. For GMM, averages were flatter and even slightly favored no explicit band-pass ($\approx$ 0.809) over 1–5 kHz ($\approx$ 0.800) and 1–3 kHz ($\approx$ 0.767), suggesting sensitivity to filter-induced feature shifts (Figures 3.12–3.13).

PCA dimensionality. Average performance peaked near PCA(8) for MLP ($\approx$ 0.875) and very low dimensions for GMM (e.g., PCA(2) $\approx$ 0.800), consistent with the different

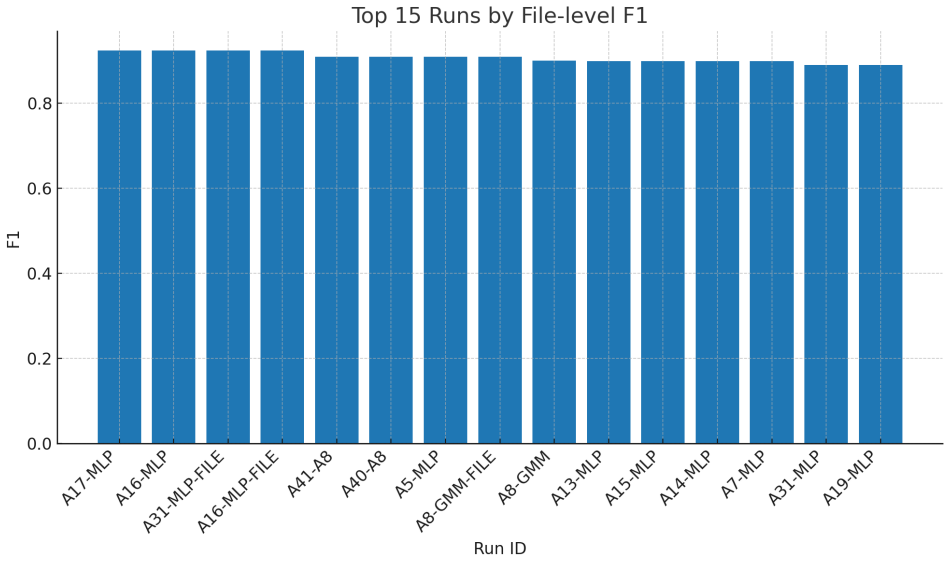


Figure 3.7: Top 15 runs by file-level F1.

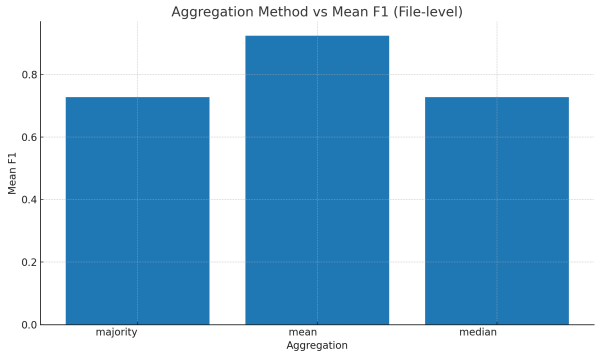


Figure 3.8: Mean F1 by aggregation method over file-level runs.

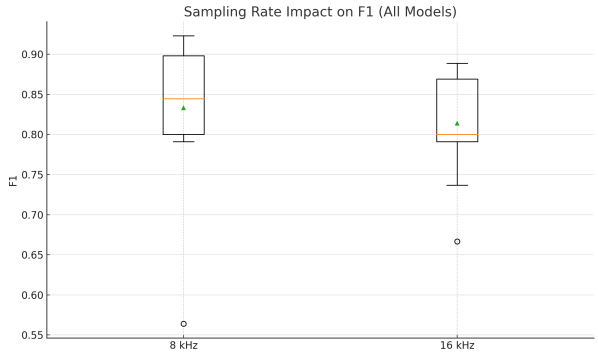


Figure 3.9: F1 distribution by sampling rate (all models).

inductive biases of neural networks and mixture models (Figures 3.14–3.15) [17].

Scaler choice. On average, MinMax scaling performed best for MLP runs in this dataset; GMM showed mixed sensitivity.

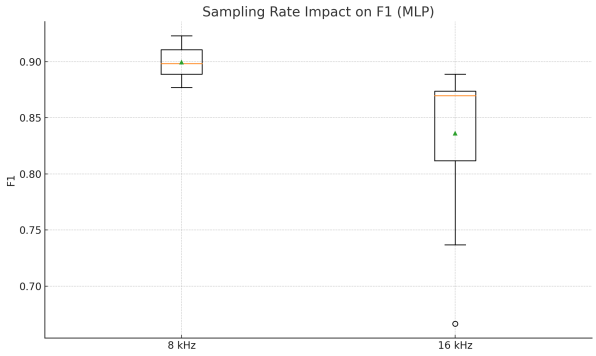


Figure 3.10: F1 distribution by sampling rate (MLP only).

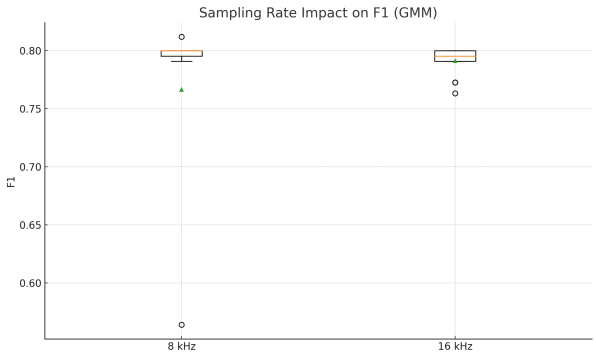


Figure 3.11: F1 distribution by sampling rate (GMM only).

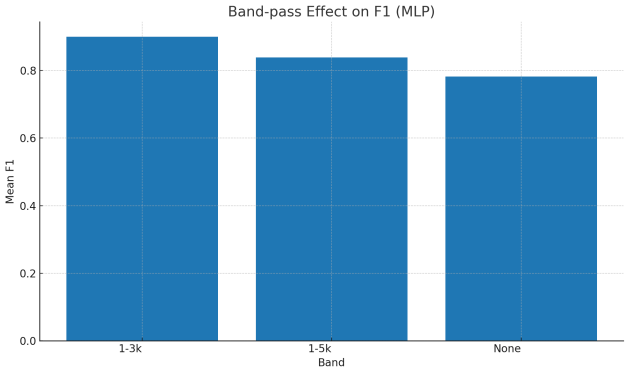


Figure 3.12: Band-pass effect on mean F1 (MLP).

– – –

Segment vs file-level. Where both were measured, file-level aggregation increased F1 relative to segment-level with a positive median ΔF1. The largest gains (top 20) are shown in Figure 3.16; distributions appear in Figure 3.17.

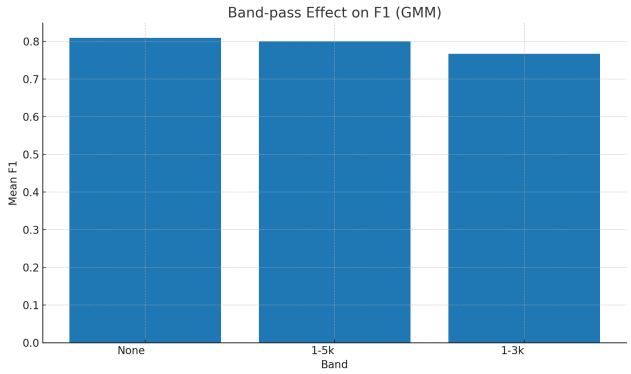


Figure 3.13: Band-pass effect on mean F1 (GMM).

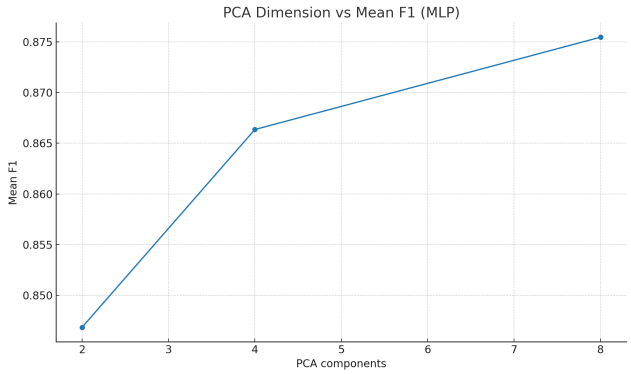


Figure 3.14: Mean F1 vs PCA components (MLP).

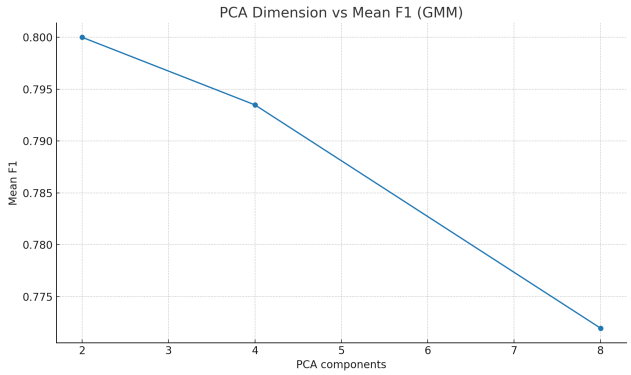


Figure 3.15: Mean F1 vs PCA components (GMM).

3.5.3 Top and Bottom Performers

Table 3.1 lists the strongest runs (one best per ID) by file-F1. Table 3.2 summarizes the lowest-performing configurations (from `ablation_clean.csv`).

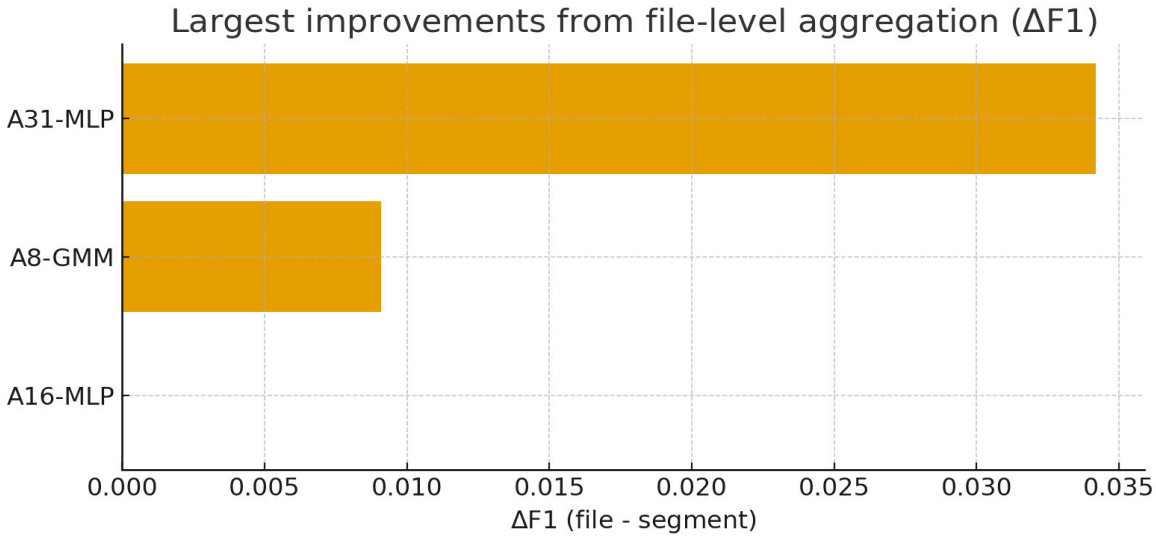


Figure 3.16: Largest improvements from file-level aggregation (ΔF1).

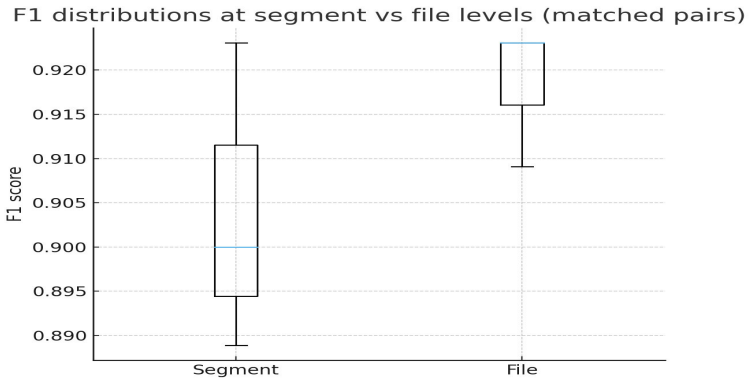


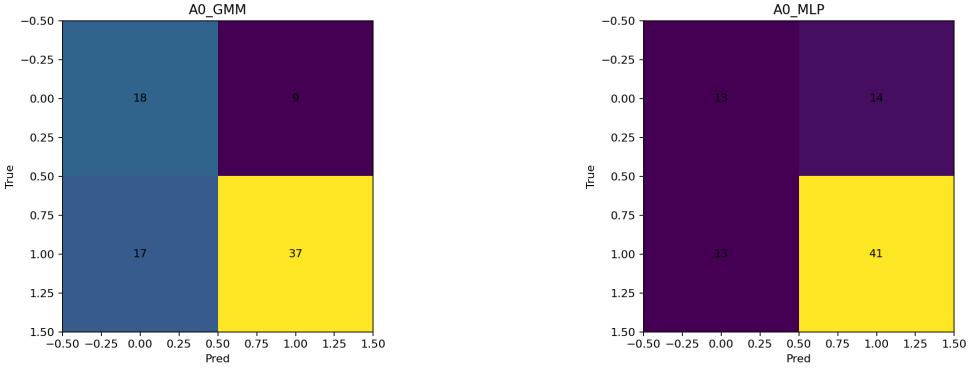
Figure 3.17: F1 distributions at segment vs file levels (where both available).

Table 3.1: Top configurations by file-level F1 (one best run per ID).

ID	**Pipeline (abridged)**	**F1**	**Acc**
A16-MLP	SR 8 kHz → BP 1–3.5 kHz → Hilbert env + log → PCA(8) → MinMax → MLP	0.923	0.889
A31-MLP-FILE	Per-file aggregation (env+log@16 kHz, mean-prob)	0.923	0.889
A17-MLP	SR 8 kHz → BP 1–3.5 kHz → env+log → MinMax + PCA(8) → MLP	0.923	0.889
A16-MLP-FILE	Per-file aggregation of A16 (mean-prob)	0.923	0.889
A40-A8 (GMM-file)	Aggregation_majority on A8	0.909	0.889
B2-A8 (GMM-file)	Resegment_B2 (A8 base)	0.909	0.889
A41-A8 (GMM-file)	Aggregation_median on A8	0.909	0.889
A8-GMM-FILE	Per-file aggregation of A8 (BP 1–5 kHz)	0.909	0.889
A5-MLP	HPF + pre-emphasis + notch@50 Hz → MLP	0.909	0.889
A8-GMM	BP 1–5 kHz → GMM	0.900	0.877

Table 3.2: Lowest file-level F1 configurations (from `ablation_clean.csv`).

ID	Pipeline (abridged)	F1	Acc
A41-A16 (MLP-file)	Aggregation_median on A16	0.545	0.444
A40-A16 (MLP-file)	Aggregation_majority on A16	0.545	0.444
A15-GMM	SR 8 kHz → BP 1–3.5 kHz → Hilbert + log + PCA(8) + fixed thr (95% keep) → GMM	0.564	0.580
A28-MLP	SR 16 kHz → BP 1–5 kHz → Hilbert + log + **no PCA** + RobustScaler → MLP	0.667	0.556
A9-MLP	Spectral whitening (FFT) → MLP	0.673	0.568



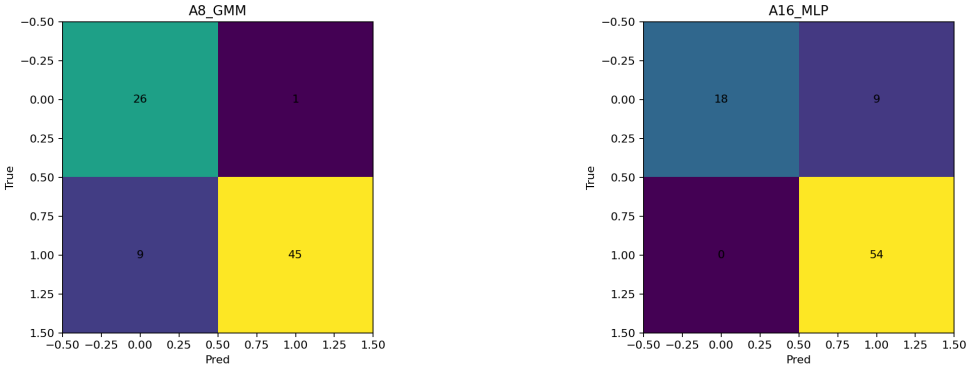
(a) A0 – GMM (b) A0 – MLP

Figure 3.18: Baselines: comparison of class-wise performance (A0).

3.5.4 Representative Confusion Matrices (Grouped)

Baselines and Early Variants

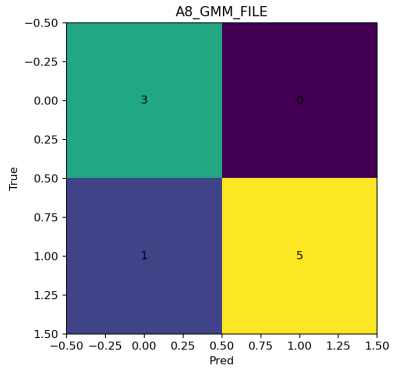
Effect of Band-pass + Envelope + Log



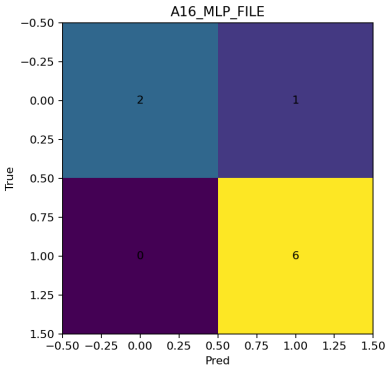
(a) A8 – GMM (b) A16 – MLP

Figure 3.19: Representative improvements with BP+Hilbert(+log) pipelines.

Per-File Aggregation Examples



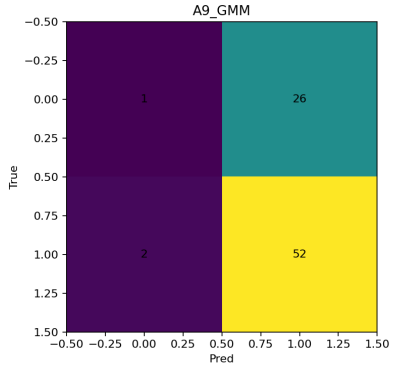
(a) A8 – GMM (file-level)



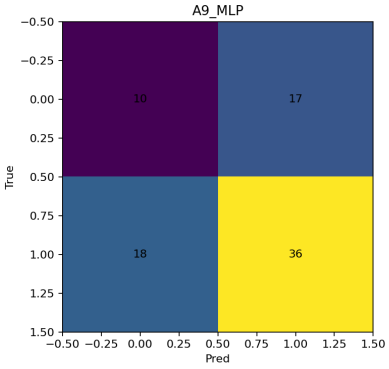
(b) A16 – MLP (file-level)

Figure 3.20: Per-file aggregation reduces frame-level volatility and improves stability.

Challenging Settings



(a) A9 – GMM



(b) A9 – MLP

Figure 3.21: Parameter changes that increased FP (GMM) and FN (MLP).

3.6 Discussion

Three robust patterns were observed:

1) **Preprocessing dominates.** Band-pass targeting 1–3.5 kHz or 1–5 kHz, Hilbert envelope, and log compression consistently improved separability (Figures 3.12, 3.13, 3.3), consistent with the impulsive/AM nature of bearing faults [8, 16].

2) **Aggregation matters.** Mean aggregation outperformed median/majority in this dataset (Figure 3.8), indicating that averaging soft probabilities preserves stable class tendencies better than vote-based rules.

3) **Model behavior differs.** MLPs benefited from narrow-band BP (1–3.5 kHz), envelope+log, and PCA(8), whereas GMMs preferred gentler dimensionality (e.g., PCA(2)) and were more sensitive to band-pass choices. Poor scaling/PCA often biased GMMs toward a single class (high FP), whereas MLPs under suboptimal preprocessing increased FN rather than collapsing.

From an embedded perspective, compact pipelines using BP+Hilbert(+log), PCA(8), and small MLPs (or carefully tuned GMMs with per-file aggregation) align with ESP32-class compute/memory constraints and TinyML practice [3, 4].

3.7 Reproducibility and Availability

All scripts, CSV summaries (`ablation_clean.csv`, `ablation_top10.csv`, `ablation_top_by_model` and confusion matrices are archived and will be made available in the SES Lab GitLab repository. Figure assets included here (`timeline_A0_A41.png`, `top3_fileF1.png`, `aggregation_compare.png`, `b0_b1_b2_compare.png`, `best_of_table.png`, and generated plots in `figs_generated/`) correspond to the experiments reported.

Chapter 4

On-Device Inference on ESP32 (M5Stack Core2)

4.1 Why A8→GMM for Embedded

From the offline study, the A8 pipeline with a Gaussian Mixture Model (GMM) ranked among the most robust segment/file classifiers while remaining lightweight. For embedded use, GMM inference reduces to linear algebra with diagonal covariances and log-likelihood sums, which is efficient in both memory and compute. The existing export of *scaler*, *PCA*, and *GMM* parameters made A8→GMM the most feasible candidate for ESP32 deployment.

Device/firmware. The target platform was an **M5Stack Core2** (ESP32) running MicroPython v1.20.0-53. The build included `ulab` (NumPy-like arrays) but did not provide `ulab.fft`; therefore spectral features used a Python DFT fallback on-device. Model assets and helpers were placed under:

- `/models/A8_GMM/preprocess.json` (scaler & PCA)
- `/models/A8_GMM/gmm.json` (weights, means, diagonal covariances)
- `/feat_a8.py`, `/model_rt_gmm.py`, `/time_gmm.py`

4.2 I/O Mode: WAV From Flash (Not Live Mic)

Live audio capture on the Core2 microphone was not used. Instead, two **pre-split** WAV files stored in flash were evaluated:

- `/wav/healthy.wav` (healthy segment)

- `/wav/anomaly.wav` (faulty segment)

Both were mono, 16-bit PCM at 16 kHz, aligned to the PC experiments. This exercised the model identically to the offline pipeline while avoiding I^2S capture complexity.

4.3 Feature Stack Implemented On-Device (A8)

A8 computes **1–5 kHz band-power** features over short frames:

- Frame length: typically 256 samples (≈16 ms) with a Hann window (timing tests also used 128/512).
- Spectrum: 1–5 kHz divided into 25 equal-width bands; integrate power per band.
- Output: a 25-D feature vector per frame.

Because `ulab.fft` was unavailable in this firmware, a Python DFT was used, with complexity $O(N^2)$; a native FFT would reduce this to $O(N \log N)$ [10, 19].

4.4 Preprocess + Classifier Runtime

The exported **preprocess** applies *z-score* normalization and **PCA(4)** (25→4 D). The **GMM** uses diagonal covariance; a fixed log-likelihood threshold gives the binary decision. On the device, *GMM-only* (preprocess+PCA+GMM) calls ran in ∼64 ms (n=500) with several megabytes of free RAM. End-to-end including A8 features (frame N=512, n=20) required ∼11.2 s per frame for feature extraction plus ∼64 ms for the classifier. The feature extractor is thus the bottleneck; the GMM stage itself is fast enough for near-real-time once features are available.

4.5 What Worked and What Did Not

The **A8→GMM** path worked end-to-end from WAVs and produced stable log-likelihood traces and file-level decisions. Pipelines requiring heavy FFT work, larger MLPs, or higher-dimensional features did not execute reliably within MicroPython memory/time constraints on this firmware.

4.6 How It Was Run (Commands and Code)

mpremote examples.

- `mpremote connect COM7 exec "import time_gmm as t; t.infer_only(500)"` (GMM-only timing)
- `mpremote connect COM7 exec "import time_gmm as t; t.end_to_end(512, 20)"` (A8+GMM timing)
- `mpremote connect COM7 exec "import a8_eval_wav_gmm as t; t.run_eval(N=128, hop=128)"` (WAV evaluation)

Minimal on-device inference (MicroPython).

```
# file: run_wav_a8_gmm.py (MicroPython on Core2)
import json, math
from ulab import numpy as np
import wavtools # read mono 16k PCM -> np.array
import feat_a8 # A8 features (DFT fallback inside)

# --- Load preprocess and GMM params exported from PC ---
pp = json.load(open('/models/A8_GMM/preprocess.json')) # scaler + PCA
    (4)
gm = json.load(open('/models/A8_GMM/gmm.json')) # w, mu, diag cov, thr

sc_mean = np.array(pp['mean'])
sc_scale = np.array(pp['scale'])
C = np.array(pp['pca']['C']) # (4,25)
mu_pca = np.array(pp['pca']['mu']) # (25,)

gmm_w = np.array(gm['w'])
gmm_mu = np.array(gm['mu'])
gmm_cv = np.array(gm['cov'])
thr = gm['thr']

def zscore(x): return (x - sc_mean) / sc_scale
def pca4(x): return (C @ (x - mu_pca))

def logpdf_diag(x, w, mu, cov):
    acc = -1e9
    for k in range(w.shape[0]):
        d = x - mu[k]
        q = -0.5 * np.sum(d*d / cov[k] + np.log(2*math.pi*cov[k]))
        acc = np.logaddexp(acc, np.log(w[k]) + q)
    return acc
```

```
def classify_frame(feat25):
    x4 = pca4(zscore(feat25))
    ll = logpdf_diag(x4, gmm_w, gmm_mu, gmm_cv)
    return (1 if ll >= thr else 0), float(ll)

sig = wavtools.read_mono16k('/wav/healthy.wav') # or '/wav/anomaly.wav'
F = feat_a8.extract(sig, fs=16000, N=256, hop=128) # -> (T,25)

labels, lls = [], []
for t in range(F.shape[0]):
    y, ll = classify_frame(F[t])
    labels.append(y); lls.append(ll)

med_ll = sorted(lls)[len(lls)//2]
y_file = (1 if med_ll >= thr else 0)
print('T frames:', len(F), 'file_pred:', y_file, 'median_ll:', med_ll)
```

Listing 4.1: Device inference: load WAV from flash, extract A8, preprocess, run GMM.

Notes: (i) `feat_a8.extract` applies windowing and band integration; (ii) a native FFT (`ulab.fft` or C) would reduce latency substantially [19].

4.7 Comparison to Vibration Pipeline

The vibration pipeline on ESP32 benefits from band-limited accelerometer streams and time-domain features that avoid heavy FFTs. In contrast, audio requires reliable spectral estimation; without a native FFT, the Python DFT dominates runtime. Moreover, the Ottawa dataset was recorded with a laboratory-grade microphone whose linearity/noise floor exceed typical ESP32 MEMS microphones, implying a sensor-quality gap for live on-device audio capture.

4.8 Custom MicroPython Firmware: Rebuild, Flashing, and Verification (M5Stack Core2)

4.8.1 Target, Versions, and Goal

A custom MicroPython image was rebuilt for the Core2 (ESP32, PSRAM) based on MicroPython v1.20.0 (board `CORE2/spiram-16m`), using ESP-IDF 4.4 and the `xtensa-esp32-elf-gcc` 8.4.0 toolchain, to include required modules (`ulab`, `ubluetooth`,

uasyncio, network, machine, esp). A single merged image was produced for flashing with esptool.

4.8.2 Key Pitfall and Resolution (ESP-IDF Mismatch)

A CMake failure due to component resolution (e.g., esp_adc_cal) was resolved by using the ESP-IDF 4.4 toolchain appropriate for the Core2 port and exporting its environment before the build [20].

4.8.3 Builtin Module Registration

For optional C modules, MicroPython v1.20 expects the two-argument MP_REGISTER_MODULE macro form in the board build system; mismatched signatures can yield link-time errors [21].

4.8.4 Build, Merge, and Flash

Firmware was built with idf.py for CORE2, then merged using esptool to allow flashing from offset 0x0 (bootloader + partitions + image). On Linux, the board was flashed over a serial device (e.g., /dev/ttyACM0) at 460,800 baud after ensuring proper permissions [20, 22].

Command sketch (verbatim).

```
idf.py -DMICROPY_BOARD=CORE2 build

python3 -m esptool --chip esp32 merge_bin -o micropython-thonny.bin \
  --flash_mode dio --flash_freq 40m --flash_size 4MB \
  0x1000  build/bootloader/bootloader.bin \
  0x8000  build/partition_table/partition-table.bin \
  0x10000 build/micropython.bin

python3 -m esptool --chip esp32 --port /dev/ttyACM0 --baud 460800 \
  write_flash -z --flash_size keep 0x0 micropython-thonny.bin
```

4.8.5 Implication for Acceleration with emlearn

To accelerate the A8 features and broaden model support, a practical approach is to move to MicroPython 1.25+ and use emlearn as an external module (FFT/IIR and tree/CNN inference in C) rather than grafting it into older custom firmware [23]. This keeps UI/control in Python while delegating DSP/ML kernels to efficient C backends.

Chapter 5

Conclusion

This thesis investigated whether *acoustic* signals can support reliable bearing fault detection under the constraints of resource-limited microcontrollers. A systematic ablation study was conducted to identify effective combinations of preprocessing, features, dimensionality reduction, models, and aggregation; the most promising pipelines were then exercised on an ESP32-class device (M5Stack Core2) to assess embedded feasibility. The work adapts the SES laboratory's vibration-centric Predictive Diagnostics and Maintenance System (PDMS) to the audio modality and documents the engineering trade-offs end-to-end.

Summary of Findings

- **Front-end efficiency is decisive.** Narrow band-pass filtering in the ∼1–5 kHz region combined with envelope and logarithmic compression produced compact, stable features for binary health classification. PCA with a small target dimension ($k \leq 8$) preserved discriminability while reducing model and memory footprint.
- **Lightweight models suffice.** On this dataset, small GMM and MLP classifiers operating on low-dimensional features achieved high F1 and accuracy. Segment-level variability was further reduced by file-level aggregation (median log-likelihood, mean probability, or majority vote), which consistently improved stability.
- **Embedded feasibility hinges on DSP kernels.** On Core2 with MicroPython v1.20, the GMM stage executed in tens of milliseconds per call, whereas feature extraction dominated latency due to the absence of a native FFT. Model inference was therefore feasible, but real-time operation requires an accelerated spectral front-end.
- **Sensor quality matters.** The reference audio was recorded with a laboratory-grade microphone; this is superior to low-cost MEMS microphones typically paired with

ESP32 modules. The gap justifies file-based evaluation during ablation and motivates careful hardware selection for live capture.

Answers to the Research Questions

RQ1 (What design choices matter most?). Pipelines using a targeted band-pass, envelope+log compression, standardization, and PCA($k \leq 8$) with either GMM or a small MLP delivered the best discriminability. In contrast, feature stacks without compression or with overly broad spectral windows underperformed.

RQ2 (Effect of aggregation). Per-file aggregation reduced variance and corrected many segment-level errors. Median log-likelihood and mean-probability aggregation provided the most consistent gains, improving both F1 and perceived robustness.

RQ3 (Embedded practicality). Classifier stages (PCA+GMM/MLP) are practical on ESP32; the bottleneck is spectral feature extraction without a native FFT. With an `ulab.fft` or C-based FFT/IIR path, the front-end cost is expected to drop by orders of magnitude, enabling streaming operation.

RQ4 (Audio vs. vibration in the SES context). Vibration remains attractive for low-cost embedded deployments because it affords strong SNR with simple time-domain or envelope methods. Audio is compelling for non-contact retrofit and complementary information, provided that microphone quality and front-end compute are addressed. The present results show that audio is viable with compact features and lightweight models, and that its deployment feasibility improves markedly once efficient DSP kernels are available.

Practical Guidance

- **Recommended recipe (binary health):** SR = 8–16 kHz, band-pass ≈1–5 kHz, envelope+log compression, standardization, PCA($k \leq 8$), and a GMM or small MLP. Use file-level aggregation for decisions in batch or slow-streaming scenarios.
- **On-device pipeline:** enable a real FFT (via `ulab.fft` or a small C FFT); consider IIR pre-emphasis and notch with fixed-point coefficients; keep model parameters in contiguous arrays to minimize allocation overhead.
- **Sensing:** prefer an external I^2S MEMS microphone with known SNR/response; maintain fixed placement and distance to reduce domain shift relative to the dataset conditions.

Limitations

The study addressed binary classification under fixed operating conditions and relied on high-quality recordings for ablation. Live capture on the Core2 internal PDM microphone was not performed due to firmware support constraints. Generalization to varying speeds/loads, additional fault classes, other microphones, and different acoustics was not evaluated and constitutes the main external-validity risk.

Future Work

1. **Accelerated DSP path.** Upgrade to a MicroPython build with `ulab.fft` or integrate C-level FFT/IIR (e.g., via a supported module) to achieve real-time feature extraction.
2. **Live audio capture.** Deploy an external I^2S microphone; calibrate gain and distance; replicate the ablation with on-device recordings to measure domain shift.
3. **Multi-class extension.** Extend labels to specific fault types (inner/outer race, ball, cage) and evaluate class-wise metrics; consider cost-sensitive thresholds.
4. **Robustness studies.** Test across operating speeds/loads and in the presence of ambient noise; include augmentation (additive/room impulse responses) to harden models.
5. **Tiny inference kernels.** Explore fixed-point or quantized MLPs and compact probabilistic models; benchmark energy/latency vs. accuracy on device.
6. **Thresholding and uncertainty.** Calibrate on-device decision thresholds and investigate simple uncertainty measures (e.g., log-likelihood margins) for conservative alerts.

Closing Remarks

It has been demonstrated that audio-based bearing fault detection is compatible with the constraints of ESP32-class devices when compact features and lightweight models are used. The chief barrier to real-time operation is not the classifier but the spectral front-end; once efficient FFT/IIR support is in place, the studied pipelines become practical for embedded monitoring. The thesis thus provides an ablation-driven roadmap—from feature choices to deployment steps—for extending SES's PDMS from vibration to audio in a principled and reproducible way.

Appendix A

Confusion Matrices of Ablation Experiments

This appendix presents the confusion matrices generated during the ablation experiments for both Gaussian Mixture Model (GMM) and Multi-Layer Perceptron (MLP) classifiers. Each matrix reports:

- **True Positive (TP)** – faulty correctly predicted as faulty,
- **True Negative (TN)** – healthy correctly predicted as healthy,
- **False Positive (FP)** – healthy misclassified as faulty,
- **False Negative (FN)** – faulty misclassified as healthy.

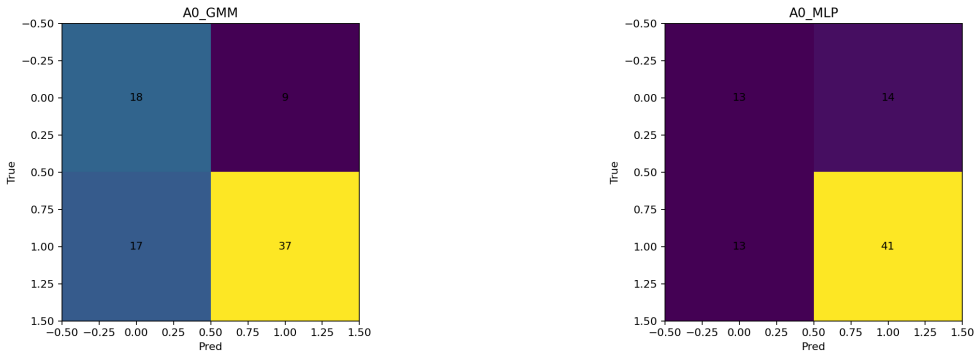
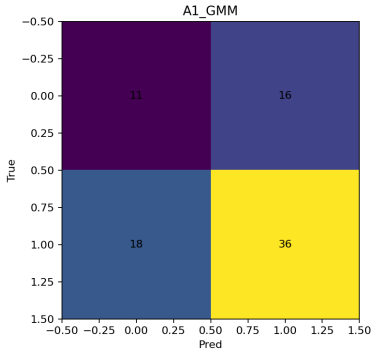


Figure A.1: A0 confusion matrices: GMM (left) and MLP (right).

Note: Additional matrices (e.g., A8_GMM_FILE, A16_MLP_FILE) can be inserted as needed using the same layout.



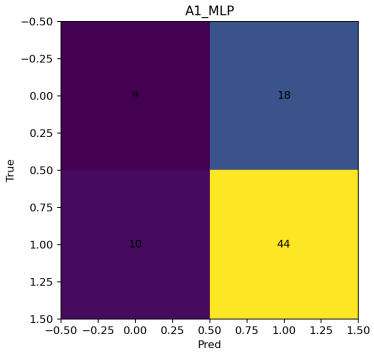
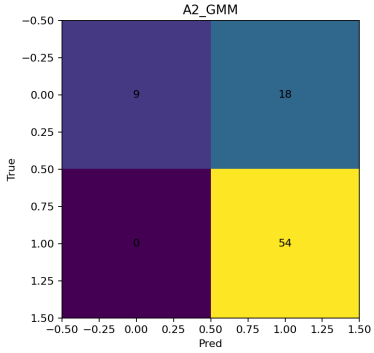


Figure A.2: A1 confusion matrices: GMM (left) and MLP (right).



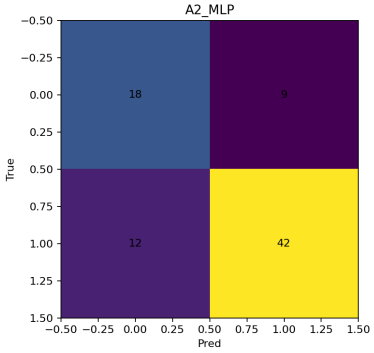
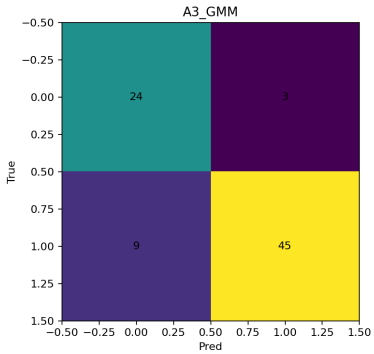


Figure A.3: A2 confusion matrices: GMM (left) and MLP (right).



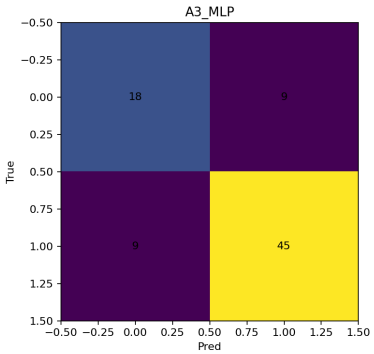
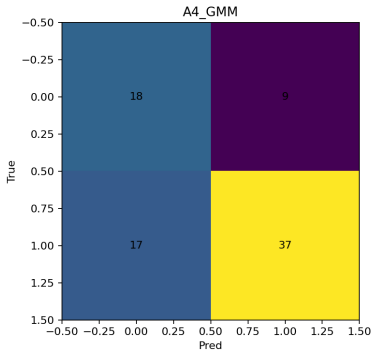


Figure A.4: A3 confusion matrices: GMM (left) and MLP (right).



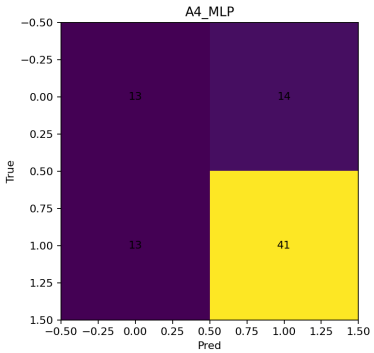
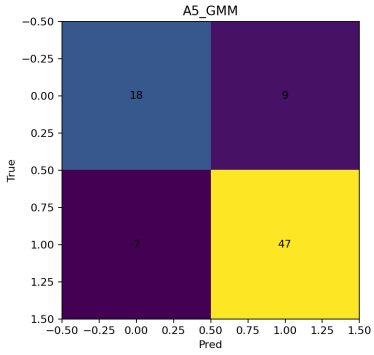


Figure A.5: A4 confusion matrices: GMM (left) and MLP (right).



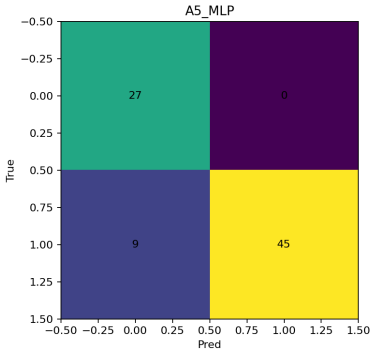
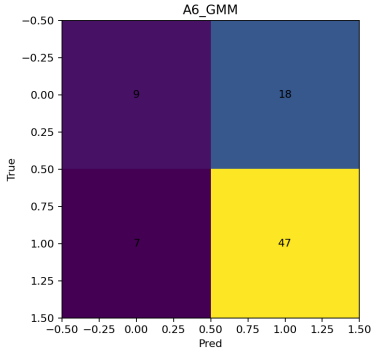


Figure A.6: A5 confusion matrices: GMM (left) and MLP (right).



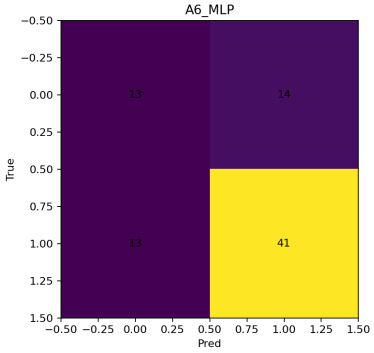
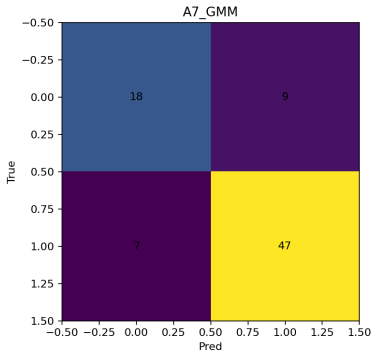


Figure A.7: A6 confusion matrices: GMM (left) and MLP (right).



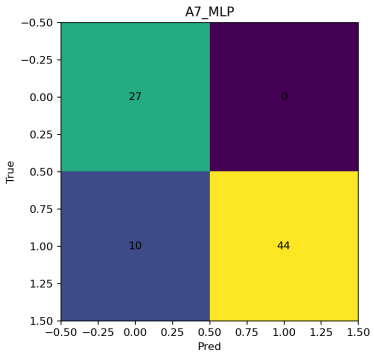
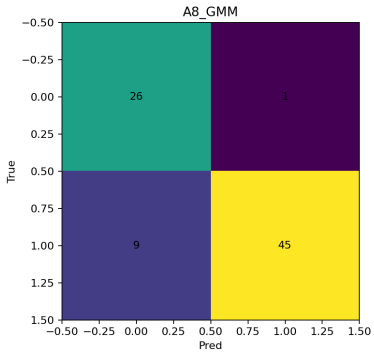


Figure A.8: A7 confusion matrices: GMM (left) and MLP (right).



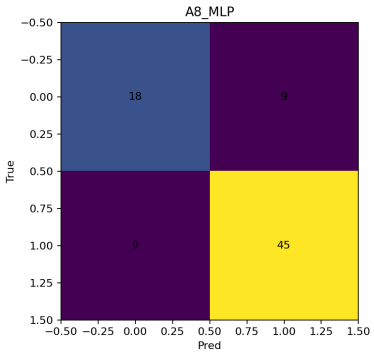
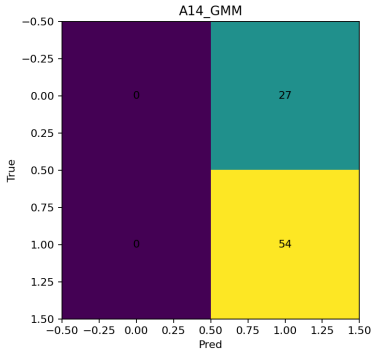


Figure A.9: A8 confusion matrices: GMM (left) and MLP (right).



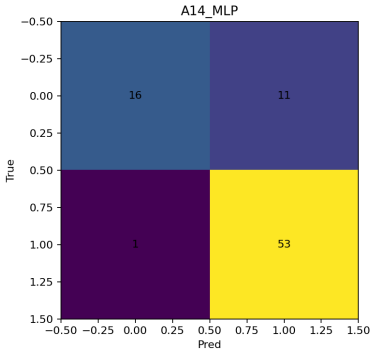
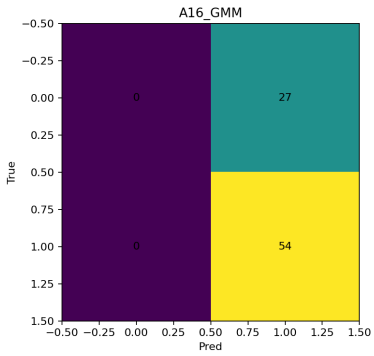


Figure A.10: A14 confusion matrices: GMM (left) and MLP (right).



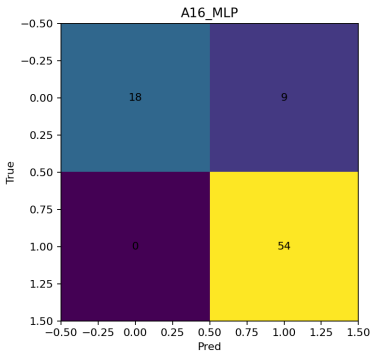
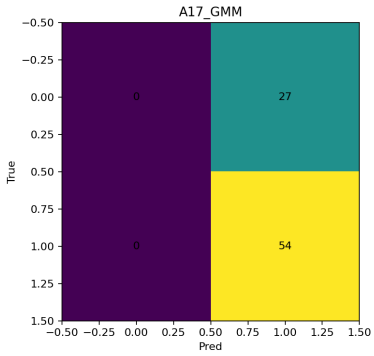


Figure A.11: A16 confusion matrices: GMM (left) and MLP (right).



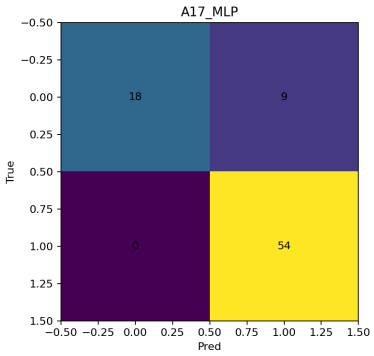
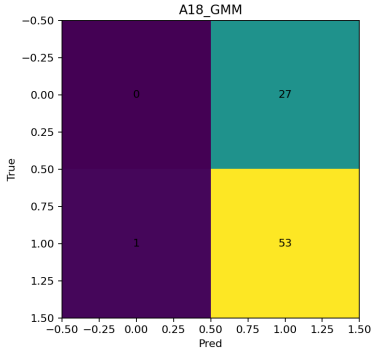


Figure A.12: A17 confusion matrices: GMM (left) and MLP (right).



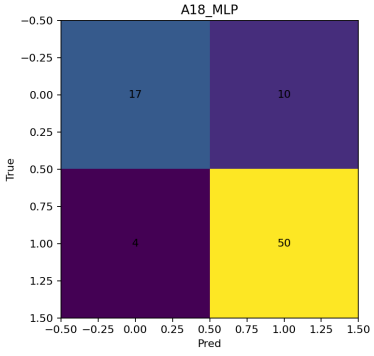
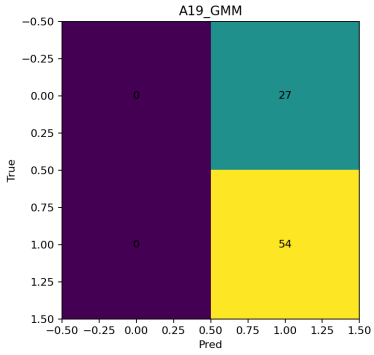


Figure A.13: A18 confusion matrices: GMM (left) and MLP (right).



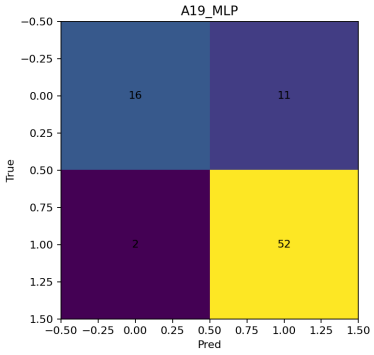
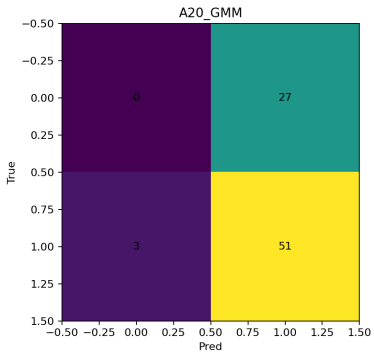


Figure A.14: A19 confusion matrices: GMM (left) and MLP (right).



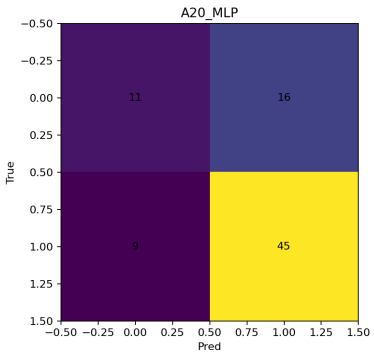
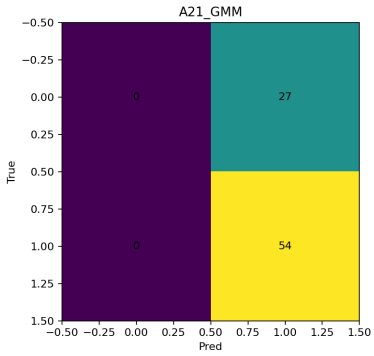


Figure A.15: A20 confusion matrices: GMM (left) and MLP (right).



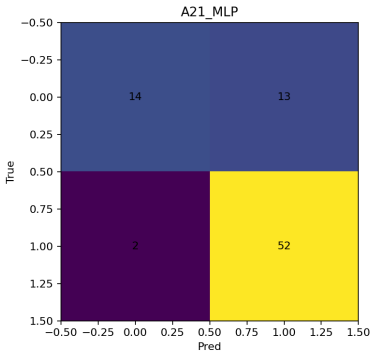
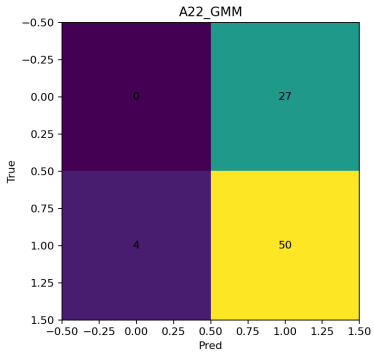


Figure A.16: A21 confusion matrices: GMM (left) and MLP (right).



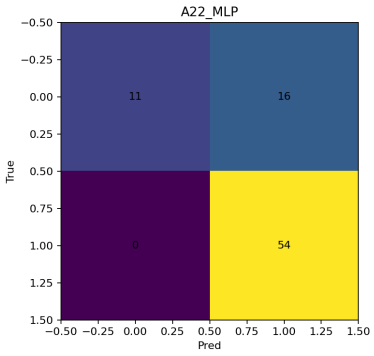


Figure A.17: A22 confusion matrices: GMM (left) and MLP (right).

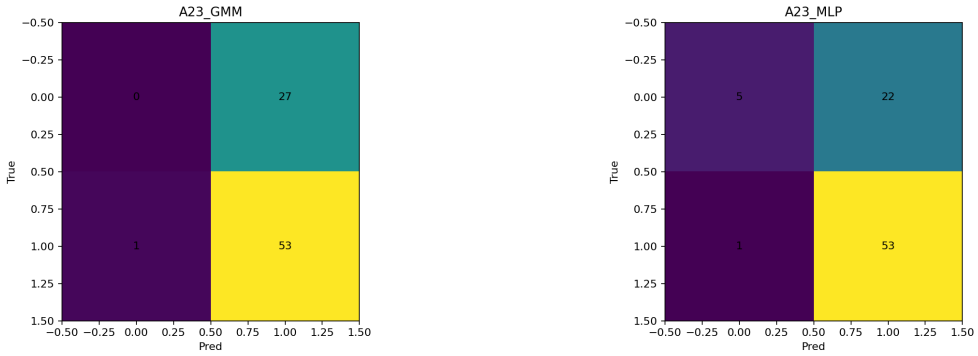


Figure A.18: A23 confusion matrices: GMM (left) and MLP (right).

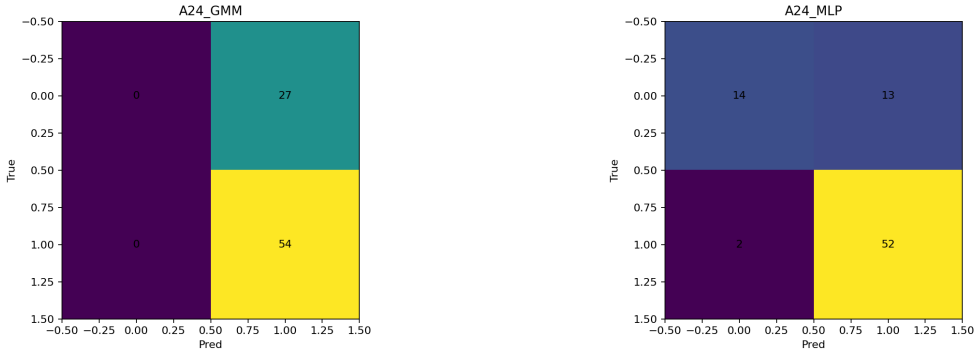


Figure A.19: A24 confusion matrices: GMM (left) and MLP (right).

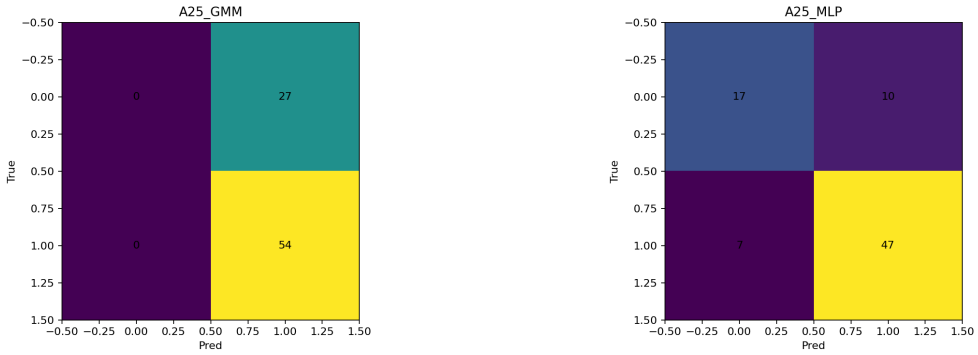


Figure A.20: A25 confusion matrices: GMM (left) and MLP (right).

Table A.1: Band-pass effect on mean F1 (MLP).

Band-pass	Mean F1
1-3k	0.900
1-5k	0.839
None	0.782

Table A.2: Band-pass effect on mean F1 (GMM).

Band-pass	Mean F1
None	0.809
1-5k	0.800
1-3k	0.767

Table A.3: Mean F1 by aggregation method (file-level runs).

Aggregation	Mean F1
majority	0.727
mean	0.923
median	0.727

Appendix B

Microphone Modalities and Capture Choice

B.1 PDM vs. PCM Microphones (Brief)

Digital MEMS microphones generally output either *pulse-density modulation* (PDM) or *pulse-code modulation* (PCM). PDM encodes the signal as a 1-bit, oversampled bitstream whose pulse density represents instantaneous amplitude; a decimation filter is then required to reconstruct band-limited PCM samples [24, 25]. PCM microphones (or I^2S digital mics with an internal ADC and decimator) deliver multi-bit samples directly and are simpler to ingest on microcontrollers when a standard I^2S interface is available [26].

B.2 M5Stack Core2 Internal Mic (PDM) and MicroPython

The M5Stack Core2 integrates an *SPM1423* digital microphone on the internal IMU board, specified by M5Stack as a **PDM** microphone [27]. At the time of this project, the ESP32 port of MicroPython did not provide a stable PDM receive path; community threads and maintainers explicitly note that PDM input is not supported (work was discussed but not merged), whereas the available `machine.I2S` interface targets standard I^2S PCM streams [28–30]. Consequently, the Core2's internal PDM mic was *not* used for capture in this work.

B.3 Reference Dataset Microphone (uOttawa) vs. M5Stack Options

The *uOttawa* bearing dataset employed a laboratory measurement microphone **PCB Piezotronics 130F20**, positioned within ~2 cm of the drive-end bearing. Reported characteristics include a sensitivity of 45 mV/Pa and an inherent noise of 29 dB; audio was acquired at 42 kHz for 10 s per file [31, 32]. By contrast, the Core2's internal SPM1423 PDM MEMS mic is specified at SNR ≈ 61.4 dB(A) [33], and a typical external I^2S MEMS option commonly used with ESP32 (e.g., **TDK/InvenSense ICS-43434**) is specified around SNR ≈ 64–65 dBA with 24-bit I^2S output [26].

Table B.1: Capture options referenced in this thesis (indicative specs).

Microphone	Interface / Key spec	Notes
PCB 130F20 (uOttawa)	Analog (via conditioner); inherent noise ~29 dB; 45 mV/Pa	Lab-grade free-field mic, placed ~2 cm from bearing; used to produce the dataset audio at 42 kHz × 10 s [31].
SPM1423 (Core2 internal)	**PDM** digital output; SNR ~61.4 dB(A)	PDM path not supported by MicroPython on ESP32 at project time; would need PDM decode or different firmware [27, 28].
ICS-43434 (external unit)	**I^2S** 24-bit PCM; SNR ~64–65 dBA	Supported via MicroPython `machine.I2S`; convenient for ESP32 capture when an external mic is acceptable [26, 29].

Rationale for file-based evaluation. Given (i) the absence of a PDM capture path in the deployed MicroPython builds, and (ii) the substantial sensor-quality gap between the uOttawa laboratory microphone and low-cost MEMS options, file-based inference from high-quality WAVs was adopted. This ensured that the ablation results reflected *model and feature* behaviour rather than being confounded by capture-device limitations. Live on-device recording remains feasible in future work using an external I^2S microphone or a firmware that natively supports PDM input.

Appendix C

Ablation Runs (Key Parameters)

This table lists each ablation run with the model tag and headline metrics. Notes are truncated to the first 60 characters for readability.

Table C.1: Ablation variants: run name, model, and headline metrics.

Run	Model	F1	Acc	Notes
A0-GMM	GMM	0.740	0.679	nan
A0-MLP	MLP	0.752	0.667	nan
A1-GMM	GMM	0.679	0.580	nan
A1-MLP	MLP	0.759	0.654	nan
A1-GMM	GMM	0.679	0.580	nan
A1-MLP	MLP	0.759	0.654	nan
A2-GMM	GMM	0.857	0.778	nan
A2-MLP	MLP	0.800	0.741	nan
A3-MLP	MLP	0.833	0.778	nan
A3-GMM	GMM	0.882	0.852	nan
A4-GMM	GMM	0.740	0.679	nan
A4-MLP	MLP	0.752	0.667	nan
A5-GMM	GMM	0.855	0.802	nan
A5-MLP	MLP	0.909	0.889	nan
A6-GMM	GMM	0.790	0.691	nan
A6-MLP	MLP	0.752	0.667	nan
A7-MLP	MLP	0.898	0.877	nan
A7-GMM	GMM	0.855	0.802	nan
A8-MLP	MLP	0.833	0.778	nan
A8-GMM-FILE	GMM-file	0.909	0.889	Per-file aggregation
A8-GMM	GMM	0.900	0.877	nan
A9-GMM	GMM	0.788	0.654	nan
A9-MLP	MLP	0.673	0.568	nan
A10-GMM	GMM	0.818	0.704	nan
A10-MLP	MLP	0.838	0.765	nan
A11-MLP	MLP	0.874	0.815	nan
A11-GMM	GMM	0.800	0.667	nan
A12-GMM	GMM	0.812	0.691	nan
A12-MLP	MLP	0.879	0.827	nan
A13-GMM	GMM	0.800	0.667	nan
A13-MLP	MLP	0.898	0.852	nan
A14-GMM	GMM	0.800	0.667	nan
A14-MLP	MLP	0.898	0.852	nan
A14-MLP	MLP	0.898	0.852	nan
A14-MLP	MLP	0.898	0.852	nan
A14-GMM	GMM	0.800	0.667	nan
A14-GMM	GMM	0.800	0.667	nan
A15-GMM	GMM	0.564	0.580	nan
A15-MLP	MLP	0.898	0.852	nan
A16-GMM	GMM	0.800	0.667	nan
A16-MLP-FILE	MLP-file	0.923	0.889	Per-file aggregation
A16-MLP	MLP	0.923	0.889	nan
A17-GMM	GMM	0.800	0.667	nan
A17-MLP	MLP	0.923	0.889	nan
A18-MLP	MLP	0.877	0.827	nan
A18-GMM	GMM	0.791	0.654	nan
A19-GMM	GMM	0.800	0.667	nan
A19-MLP	MLP	0.889	0.840	nan
A20-GMM	GMM	0.773	0.630	nan
A20-MLP	MLP	0.783	0.691	nan
A21-GMM	GMM	0.800	0.667	nan
A21-MLP	MLP	0.874	0.815	nan

Bibliography

[1] S. S. Gill, M. Golec, J. Hu, M. Xu, J. Du, H. Wu *et al.*, "Edge ai: A taxonomy, systematic review and future directions," *Cluster Computing*, 2025. [Online]. Available: https://doi.org/10.1007/s10586-024-04686-y

[2] V. Tsoukas, S. Nikolaidis, C. Papachristou *et al.*, "A review on the emerging technology of tinyml," *ACM Computing Surveys*, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3661820

[3] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, "Tensorflow lite micro: Embedded machine learning on tinyml systems," in *Proceedings of Machine Learning and Systems (MLSys)*, 2021. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2021/file/6c44dc73014d66ba49b28d483a8f8b0d-Paper.pdf

[4] C. Banbury, V. Janapa Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau, U. Thakker, A. Torrini, P. Warden *et al.*, "Mlperf tiny benchmark," *arXiv preprint arXiv:2106.07597*, 2021. [Online]. Available: https://arxiv.org/abs/2106.07597

[5] R. B. Randall and J. Antoni, "Rolling element bearing diagnostics—a tutorial," *Mechanical Systems and Signal Processing*, vol. 25, no. 2, pp. 485–520, 2011.

[6] N. Tandon and A. Choudhury, "A review of the condition monitoring and fault diagnosis of rolling element bearings using vibration analysis," *Tribology International*, vol. 32, no. 8, pp. 469–480, 1999.

[7] J. Antoni, "The spectral kurtosis: a useful tool for characterising non-stationary signals," *Mechanical Systems and Signal Processing*, vol. 20, no. 2, pp. 282–307, 2006.

[8] D. Zhang *et al.*, "Intelligent acoustic-based fault diagnosis of roller bearings: A review," *Measurement*, vol. 163, p. 107924, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0263224120301226

[9] F. Alharbi, X. Yang *et al.*, "A brief review of acoustic and vibration signal-based fault detection for belt conveyor idlers using machine learning models," *Sensors*, vol. 23, no. 3, p. 9959905, 2023, open access. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC9959905/

[10] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 2nd ed. Prentice Hall, 1999.

[11] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.

[12] J. Kiener, "Porting and evaluation of a python machine learning system for a linux-based microcontroller," Master's thesis, OTH Regensburg, 2025, accessed 2025-08-14.

[13] M. Sehri, P. Dumond, and M. Bouchard, "University of ottawa rolling-element bearing vibration and acoustic fault signature datasets operating under constant load and speed conditions," *Data in Brief*, vol. 49, 2023, cC BY 4.0, accessed 2025-08-14.

[14] K. Stiglmaier, "Anomaly detection in bearings using raw and feature-extracted data using supervised and unsupervised learning methods," Master's thesis, OTH Regensburg, 2023, accessed 2025-08-14.

[15] J. Kiener, "Evaluating feature-based supervised and unsupervised machine learning models for bearing fault detection," Master's thesis, OTH Regensburg, 2022, accessed 2025-08-14.

[16] L. R. Rabiner and R. W. Schafer, *Theory and Applications of Digital Speech Processing*. Prentice Hall, 2011.

[17] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[18] D. M. W. Powers, "Evaluation: From precision, recall and f-measure to roc, informedness, markedness and correlation," *Journal of Machine Learning Technologies*, vol. 2, no. 1, pp. 37–63, 2011, accessed 2025-08-14. [Online]. Available: https://doi.org/10.48550/arXiv.2010.16061

[19] ulab Contributors, "ulab for micropython: Api and fft notes," https://micropython-ulab.readthedocs.io/, 2025, accessed 2025-08-14.

[20] *ESP-IDF Programming Guide v4.4 (ESP32)*, https://docs.espressif.com/projects/esp-idf/en/v4.4/esp32/, Espressif Systems, 2022, accessed 2025-08-14.

[21] MicroPython Developers, “Micropython v1.20.0 documentation,” https://docs.micropython.org/en/v1.20.0/, 2023, accessed 2025-08-14.

[22] Espressif Systems, “esptool: Esp32/esp8266 serial flasher utility,” https://docs.espressif.com/projects/esptool/, 2025, accessed 2025-08-14.

[23] emlearn Contributors, “emlearn: Machine learning for microcontrollers,” https://emlearn.org/, 2025, accessed 2025-08-14.

[24] T. Kite, “Understanding pdm digital audio,” https://users.ece.utexas.edu/~bevans/courses/rtdsp/lectures/10_Data_Conversion/AP_Understanding_PDM_Digital_Audio.pdf, 2011, intro to 1-bit PDM and decimation; accessed 2025-08-14.

[25] *AN3998: PDM Audio Software Decoding on STM32 Microcontrollers*, https://www.st.com/resource/en/application_note/an3998-pdm-audio-software-decoding-on-stm32-microcontrollers-stmicroelectronics.pdf, STMicroelectronics, 2011, explains PDM → PCM decimation; accessed 2025-08-14.

[26] *ICS-43434: I^2S Digital MEMS Microphone — Datasheet*, https://invensense.tdk.com/wp-content/uploads/2016/02/DS-000069-ICS-43434-v1.2.pdf, TDK InvenSense, 2016, sNR ~64–65 dBA, 24-bit I^2S; accessed 2025-08-14.

[27] M5Stack, “M5stack core2 product documentation,” https://docs.m5stack.com/en/core/core2, 2025, accessed 2025-08-14.

[28] MicroPython Community, “Micropython discussion: “i2s pdm mode?” (esp32, pdm not supported),” https://github.com/orgs/micropython/discussions/14160, 2024, confirms lack of PDM RX support; accessed 2025-08-14.

[29] MicroPython Developers, “Micropython: `machine.I2S` (esp32) — api (technical preview),” https://docs.micropython.org/en/latest/library/machine.I2S.html, 2025, documents I^2S PCM interface (no PDM capture); accessed 2025-08-14.

[30] Seeed Studio Community, “Forum thread: “esp32-s3 sense microphone with micropython”,” https://forum.seeedstudio.com/t/esp32-s3-sense-microphone-with-micropython/275764, 2025, states PDM not supported in MicroPython, references PR; accessed 2025-08-14.

[31] M. Sehri, P. Dumond, and M. Bouchard, “University of ottawa ball-bearing vibration and acoustic fault data under constant load and speed conditions (uodsvafdc): Experimental setup,” https://papers.ssrn.com/sol3/papers.cfm?abstract_

id=4431783, 2023, preprint with hardware list; microphone model *PCB 130F20*; accessed 2025-08-14.

[32] M. Sehri and P. Dumond, "University of ottawa rolling-element dataset (raw signals)," https://data.mendeley.com/datasets/y2px5tg92h, 2023, sampling rate 42 kHz, 10 s per file; accessed 2025-08-14.

[33] M5Stack, "Unit mini pdm (spm1423) — specifications," https://docs.m5stack.com/en/unit/pdm, 2025, spec sheet with SNR for SPM1423; accessed 2025-08-14.